Final Year Project Documentation

QuickFix: On-Road Breakdown Assistance Application

By

Project ID:	**BSIT22-G20**
Kinza Yousaf	**2022-ICG-174**
Arooba Usman	**2022-ICG-168**

Under the Supervision of

Prof. Muhammad Asif

Bachelor of Science in Information Technology,

Session (2022-2026).

FACULTY OF COMPUTING &

INFORMATION TECHNOLOGY,

UNIVERSITY O F THE PUNJAB, LAHORE.

QuickFix: On-Road Breakdown Assistance Application

A Project represented to

University of the Punjab, Lahore.

In Partial fulfilment of

the requirement for the degree of

Bachelor of Science in Information Technology,

Session (2022-2026).

By

Kinza Yousaf **2022-ICG-174**

Arooba Usman **2022-ICG-168**

FACULTY OF COMPUTING &

INFORMATION TECHNOLOGY,

UNIVERSITY O F THE PUNJAB, LAHORE.

DECLARATION

We hereby declare that this software, neither the whole nor a part, was copied from any source. It is further declared that we have developed this software and accompanying report based on our efforts. If any part of this project is proven to be copied from any source or found to be a reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted for any application for any other degree or qualification of this or any other university or institute of learning.

Kinza Yousaf [076949]

Signature: _____

Arooba Usman [076962]

Signature: _____

CERTIFICATE OF APPROVAL

It is to certify that the final year project (FYP) of BSIT, session (2022-2026), the project QuickFix: On-Road Breakdown Assistance Application was developed by Kinza Yousaf (2022-ICG-174), and Arooba Usman (2022-ICG-168) under the supervision of Prof. Muhammad Asif Sb. in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Information Technology.

FYP Advisor: Prof. Muhammad Asif sb.

Advisor Signature: _____

Signatures (Faculty Advisory Committee (FAC))

	FAC1	**FAC2**
Name	**Prof. Muhammad Asif Sb.**	**Prof. Usman Ahmad Sb.**
Signature		

Head of FYP Coordination Office: Prof. Muhammad Asif Sb.

Signature: _____

Head Of Department, Information Technology: Prof. Usman Ahmad Sb.

Signature: _____ Dated: _____

Executive Summary

QuickFix - A Vehicle Breakdown Assistance is a cross-platform mobile application specially designed to instantly connect travelers with mechanics and fuel stations for immediate fuel delivery and mechanical help. Platform enables admin to approve/reject provider's application based on their verified credentials. The app offers a smooth service experience with access to finding nearby service providers, provide current service status and real-time location tracking features.

The app provides a simple and user-friendly interface where service providers such as mechanics and fuel sellers can post services with description and pricing details. Platform allow travelers to find nearby providers, browse service listing with all information. People can easily get these services and book a service request in case of any vehicle emergency. App allows both travelers to track current location on map, get their order updates instantly. People can rate the providers and submit their valuable feedbacks after completion of service.

The system is designed using modern technologies such as React Native to ensure smooth experience across all platforms, Firebase firestore for data storage and management, role-based authentication is securely handled by Firebase authentication, uses FCM for handling push notifications, Firebase cloud Functions manages all backend processes and integrating Open Street Maps for real-time location updates.

Application follows the Model-View-Presenter architectural Pattern and will developed using Object Oriented Programming Principles. This structure ensures separation of data logic from the user interface, provides scalability and maintainability. In short, QuickFix is a reliable, user-friendly service solution designed for travelers which help them to quickly overcome vehicle emergencies.

Acknowledgment

We express our gratitude to our Almighty Allah, and after that, we are thankful to our families, who allowed us to get higher education, and we became capable of creating something productive. We are also thankful to our project advisor and department faculty for their support.

Kinza Yousaf [076949]

Signature: _____

Arooba Usman [076962]

Signature: _____

Abbreviation

Table of all the abbreviations and acronyms used in your FYP, along with their respective brief description. The abbreviation list must be ordered alphabetically.

IT	Information Technology
DB	Database
PU	Punjab University
RAM	Random Access Memory
JS	JavaScript
FCM	Firebase Cloud Messaging
UI	User Interface
RN	React Native
CF	Cloud Functions

Table of Contents

Chapter 1 18

1. Introduction 19

1.1 Problem Statement 19

1.2 Problem Solution 19

1.3 Objectives of Proposed System 19

1.4 Scope 20

1.5 System Components 21

1.5.1 Feature 1: User Authentication (Customer App) 21

1.5.2 Feature 2: Home Page (Customer App) 21

1.5.3 Feature 3: Service Selection (Customer App) 22

1.5.4 Feature 4: Checkout Phase (Customer App) 22

1.5.5 Feature 5: Track Order (Customer App) 23

1.5.6 Feature 6: User Authentication (Provider App) 23

1.5.7 Feature 7: Home Screen (Provider App) 24

1.5.8 Feature 8: Order Section (Provider App) 24

1.5.9 Feature 9: Profile Section (Customer & Provider App) 25

1.5.10 Feature 10: Admin Mobile App 25

1.6 Related System Analysis/Literature Review 26

1.7 Vision Statement 26

1.8 System Limitations and Constraints 27

1.9 Project Deliverables 27

1.10 Tools and Technologies 28

1.11 Project Planning 29

1.12 Summary 29

Chapter 2 30

2. Analysis 31
2.1 User Classes and Characterization 31
2.2 Requirement Identifying Techniques 31
2.2.1 Interviews 32
2.2.2 Ethnography or Observation 32
2.2.3 Use Case and Scenarios 32
2.3 Functional Requirements 32
2.3.1 Functional Requirement 1 32
2.3.2 Functional Requirement 2 33
2.3.3 Functional Requirement 3 34
2.3.4 Functional Requirement 4 34
2.3.5 Functional Requirement 5 35
2.3.6 Functional Requirement 6 35
2.3.7 Functional Requirement 7 36
2.3.8 Functional Requirement 8 36
2.3.9 Functional Requirement 9 37
2.3.10 Functional Requirement 10 37
2.3.11 Functional Requirement 11 38
2.3.12 Functional Requirement 12 38
2.3.13 Functional Requirement 13 39
2.3.14 Functional Requirement 14 39
2.3.15 Functional Requirement 15 40
2.3.16 Functional Requirement 16 41
2.3.17 Functional Requirement 17 41
2.3.18 Functional Requirement 18 42
2.3.19 Functional Requirement 19 42

2.4 Non-Functional Requirements 43
2.5 Use Case Analysis 44
2.5.1 Use Case 1 44
2.5.2 Use Case 2 45
2.5.3 Use Case 3 46
2.5.4 Use Case 4 47
2.5.5 Use Case 5 48
2.5.6 Use Case 6 49
2.5.7 Use Case 7 50
2.5.8 Use Case 8 51
2.5.9 Use Case 9 52
2.5.10 Use Case 10 53
2.5.11 Use Case 11 54
2.5.12 Use Case 12 55
2.5.13 Use Case 13 56
2.5.14 Use Case 14 57
2.5.15 Use Case 15 59
2.5.16 Use Case 16 60
2.5.17 Use Case Diagram 61
2.6 External Interface Components 61
2.6.1 User Interface Requirements 61
2.6.2 Software Interfaces 62
2.6.3 Hardware Interfaces 63
2.6.4 Communication Interfaces 64
2.7 Summary 64
Chapter 3 65

3. System Design 66
3.1 Design Considerations 66
3.1.1 Assumptions and Dependencies 66
3.1.2 Limitations 67
3.1.3 Risks and Obstacles 67
3.2 Design Models 67
3.2.1 Class Diagram 67
3.2.2 Sequence Diagram 68
3.2.3 State Diagram 75
3.3 Architectural Design 79
3.3.1 Component Diagram 80
3.3.2 Architectural Pattern Diagram 81
3.4 Data Design 81
3.4.1 Data Dictionary 82
3.5 User Interface Design 83
3.5.1 Screen Images 83
3.5.2 Screen Objects and Actions 84
3.6 Design Decisions 84
3.7 Summary 85
Chapter 4 86
4. Implementation 87
4.1 Algorithms 87
4.1.1 Algorithm 1 – Customer Registration 87
4.1.2 Algorithm 2 – Sign In 88
4.1.3 Algorithm 3 – Search Nearby Providers 89
4.1.4 Algorithm 4 – Service Selection 90

4.1.5 Algorithm 5 – Checkout 90
4.1.6 Algorithm 6 – Request Tracking 91
4.1.7 Algorithm 7 – Provider Registration 92
4.1.8 Algorithm 8 – Service Creation 93
4.1.9 Algorithm 9 – Service Updates 94
4.1.10 Algorithm 10 – Manage Profile 95
4.1.11 Algorithm 11 – Notification & Feedback 96
4.2 External APIs/SDKs 97
4.3 Code Repository 97
4.4 Summary 97
Chapter 5 98
5. Testing and Evaluation 99
5.1 Unit Testing 99
5.2 Functional Testing 106
5.3 Integration Testing 111
5.4 Performance Testing 115
5.5 Summary 117
Chapter 6 118
6. System Conversion 119
6.1 Conversion Method 119
6.1.1 Pilot Conversion 119
6.2 Deployment 120
6.2.1 Deployment Steps: 120
6.2.2 Data Conversion 120
6.2.3 Training 121
6.3 Post Deployment Testing 122

6.3.1 Post Deployment Testing Steps 122
6.4 Challenges and Solutions 122
6.5 Summary 123
Chapter 7 124
7. Introduction 125
7.1 Evaluation 125
7.2 Traceability Matrix 125
7.3 Conclusion 128
7.4 Future Works 128
References 129
Appendix A: Use Case Description 130
Appendix B: General Coding Standards & Guidelines 131
Variable Naming Conventions 131
Boolean Naming 131
Variable Declaration 131
Function Naming 131
Commenting Style 131
Indentation and Code Structure 132
File and Folder Naming 132
Separation of Concerns 132
Scoping and Accessibility 132
Cohesion and Coupling 132
DRY Principle (Don’t Repeat Yourself) 132
Single Responsibility Principle (SRP) 133
Extensibility and Maintainability 133
Exception Handling and Logging 133

Appendix C: Application Prototype 134
Prototype Overview 134
Tool Used for Prototype 134
Screenshots of Key Interfaces 134
Navigation Flow 135
Responsive Design 135

List of Tables

Table 1 Related System Analysis 26
Table 2 Tools and Technologies 28
Table 3 User Class and Characterization 31
Table 4 Functional Requirement 1 32
Table 5 Functional Requirement 2 33
Table 6 Functional Requirement 3 34
Table 7 Functional Requirement 4 34
Table 8 Functional Requirement 5 35
Table 9 Functional Requirement 6 35
Table 10 Functional Requirement 7 36
Table 11 Functional Requirement 8 36
Table 12 Functional Requirement 9 37
Table 13 Functional Requirement 10 37
Table 14 Functional Requirement 11 38
Table 15 Functional Requirement 12 38
Table 16 Functional Requirement 13 39
Table 17 Functional Requirement 14 39
Table 18 Functional Requirement 15 40
Table 19 Functional Requirement 16 41
Table 20 Functional Requirement 17 41
Table 21 Functional Requirement 18 42
Table 22 Functional Requirement 19 42
Table 23 Non-Functional Requirements 43
Table 24 Use Case 1 44
Table 25 Use Case 2 45
Table 26 Use Case 3 46
Table 27 Use Case 4 47
Table 28 Use Case 5 48
Table 29 Use Case 6 49
Table 30 Use Case 7 50
Table 31 Use Case 8 51
Table 32 Use Case 9 52

Table 33 Use Case 10 53
Table 34 Use Case 11 54
Table 35 Use Case 12 55
Table 36 Use Case 13 56
Table 37 Use Case14 57
Table 38 Use Case 15 59
Table 39 Use Case 16 60
Table 40 User Interface Components 61
Table 41 Software Interfaces 63
Table 42 Hardware Interfaces 63
Table 43 Communication Interfaces 64
Table 44 Risks and Obstacles 67
Table 45 Data Dictionary 82
Table 46 Unit Test Case UT1 99
Table 47 Unit Test Case UT2 100
Table 48 Unit Test Case UT3 101
Table 49 Unit Test Case UT4 102
Table 50 Unit Test Case UT5 103
Table 51 Unit Test Case UT6 104
Table 52 Unit Test Case UT7 104
Table 53 Unit Test Case UT8 105
Table 54 Functional Test Case FT1 106
Table 55 Functional Test Case FT2 107
Table 56 Functional Test Case FT3 108
Table 57 Functional Test Case FT4 109
Table 58 Functional Test Case FT5 109
Table 59 Functional Test Case FT6 110
Table 60 Integration Test Case IT1 111
Table 61 Integration Test Case IT2 112
Table 62 Integration Test Case IT3 113
Table 63 Integration Test Case IT4 113
Table 64 Integration Test Case IT5 114
Table 65 Performance Testing Results 115
Table 66 Performance Testing PT1 117

Table 67 Deployment Steps 120
Table 68 Data Conversion 120
Table 69 Training Instruction 1 121
Table 70 Training Instruction 2 121
Table 71 Post Deployment Testing Steps 122
Table 72 Challenges and Solutions 123
Table 73 Progress Table 125
Table 74 Traceability Matrix Table 125
Table 75 Use Case Description 130

List of Figures

Figure 1 Use Case Diagram 61
Figure 2 Class Diagram 68
Figure 3 Sequence Diagram 1 69
Figure 4 Sequence Diagram 2 70
Figure 5 Sequence Diagram 3 71
Figure 6 Sequence Diagram 4 72
Figure 7 Sequence Diagram 5 73
Figure 8 Sequence Diagram 6 74
Figure 9 Sequence Diagram 7 75
Figure 10 State Diagram 2 76
Figure 11 State Diagram 1 76
Figure 12 State Diagram 3 77
Figure 13 State Diagram 4 77
Figure 14 State Diagram 6 78
Figure 15 State Diagram 5 78
Figure 16 State Diagram 7 79
Figure 17 Component Diagram 80
Figure 18 Architectural Pattern Diagram 81
Figure 19 Firestore Hierarchy 81
Figure 20 Screen Images 83
Figure 21 Screen Objects and Actions 84

Chapter 1

Introduction

1. Introduction

This Chapter brings the emerging trend of on-road vehicle services along with the problem for travelers, who are facing the on-road emergencies in case of fuel shortage or any other mechanical breakdown like tyre burst, oil change and brake inspection etc. Our modern and user-friendly aims to fulfil these needs with interactive features, with the help of several tools and technologies, along with constraints, limitations and enhancement listed in the sections discussed below.

1.1 Problem Statement

People often experience common problems such as vehicle breakdown or fuel shortage. The difficulty arise where they have to access immediate mechanical and fuel service. Traditional methods for roadside help are often time-consuming and inefficient. Consequently, people face lengthy delays which leave them anxious and waste their valuable time. The average waiting time for emergency vehicle service is around 45 to 60 minutes. This delay causes unnecessary stress for people and financial loss for service providers. So, there is a clear need for a modern and user-friendly solution that instantly connect people with service providers. People can easily get their issues fixed like vehicle breakdown and fuel shortage.

1.2 Problem Solution

We are developing a cross-platform mobile application named “QuickFix” designed to provide quick roadside services. The app makes it easy for anyone to get immediate help in case of fuel shortages and mechanical breakdowns. QuickFix connects you directly with nearby service providers in real-time. By confirming your request through the app, it tells you exactly who is coming and how many minutes away they are. Instead of worrying on the side of road, people can relax knowing help is already on the way.

1.3 Objectives of Proposed System

1. Build a mobile application that quickly connect people to nearby mechanics and fuel stations for roadside assistance.
2. Role-based authentication to manage access for customers, providers and admin using firebase.

3. Integrate maps for live tracking using OpenStreetMaps.
4. System for providing current service status updates.
5. Notification system to instantly notify new requests and status changes using FCM.
6. Rating system to collect feedbacks and enhancing provider's performance.
7. Profile section to view and update personal details.
8. System for admin to accept new requests, deleting/suspending accounts and viewing feedbacks.
9. Successfully launch the app to major platforms like Android and Ios.

1.4 Scope

1. The scope of this project is to develop a cross-platform mobile application specially designed for travelers who face on-road vehicle breakdown emergencies.
2. The following will be stakeholders:
3. Customers
4. Mechanics
5. Fuel Stations
6. Admin
7. Using OpenStreetMap to show the real-time location of both the customer and the provider.
8. Secure login and Sign up for all users via Firebase Authentication.
9. For development, React Native will be used for cross-platform UI and core logic, JS for frontend functionality and Firebase Cloud Functions for Backend logic.
10. Firebase Firestore will be used to manage data storage.
11. Instant notifications is handled by Firebase Cloud Messaging.
12. Separate Admin Panel to manage users, verify providers and monitoring system activity.

Limitations

1. The system will not include online payment process.
2. System will not provide offline feature.
3. The system will not include any chat messaging.

1.5 System Components

The app will have four modules: (1) Customer Mobile App, (2) Mechanic Mobile App, (3) Fuel Station Mobile app, (4) Admin Mobile App. End-users must have an active internet connection to send out new service request and receive live updates. Collectively, the app will have the following features.

1.5.1 Feature 1: User Authentication (Customer App)

The list of features this module will have and how it will be used is as follows.

1. A splash screen to initialize or load user-specific data if authenticated, else redirected to the authorization screen. The splash screen will check if there is user logged in or not using Firebase Authentication. If user is logged in then user will be redirected to home screen.
2. The Authorization page will have two options for user, to log in, to sign up.
3. The Sign-Up screen will ask user to enter his personal details like email, full name, address, picture and password. After validating the information provided, the user will be created and redirected to the Home Screen.
4. The Sign In screen will ask user to enter his email and password. After validation & verification the user will be redirected to home screen. Login screen will also have option for forgot password with email verification.
5. The entire authentication module is implemented using Firebase Authentication.

1.5.2 Feature 2: Home Page (Customer App)

Once a user login, they are taken to the main customer dashboard.

1. Upon successful authentication, the home screen features an interactive map, integrated by OpenStreetMap and React Native Libraries.
2. From splash screen, when user lands on the home screen, app ask user to enable his Device’s Location.
3. Search option is placed at the top of screen, allow user to manually look for nearby service providers such as Mechanic and Fuel Sellers.
4. A clear navigation bar at the bottom provides quick access to the three essential areas. (1) Home, (Orders) and (Profile).

5. The search logic is handled by Firebase Cloud Functions to filter the nearest providers.

1.5.3 Feature 3: Service Selection (Customer App)

At this point, user select the needed services and make a request.

1. When user searches on search bar, app instantly switches to a new page that list all mechanics and fuel stations currently available in your area.
2. Customer can easily compare different providers and pick the best one based on overall ratings and orders.
3. By clicking a specific provider, app opens a dedicated page that shows the full details, service listings with their description and fixed prices.
4. Users can select exactly which services they need by simply ticking the checkboxes next to each item.
5. Once the user picked what they need, they tap on “Next” button that takes you to the final step to confirm your request.
6. Firebase Firestore will be used to fetch provider’s data.
7. JavaScript logic filters the list and Firebase syncs the rating data.

1.5.4 Feature 4: Checkout Phase (Customer App)

Final phase of confirming the order is given as:

1. App takes the user to a summary page when user click “Proceed to Checkout”, it breakdown all costs including service fee.
2. Once everything looks correct, user simply tap “Confirm Order” button to officially place the service request.
3. The request alert is sent directly to the provider’s phone so they can see request order immediately.
4. Firebase Cloud Messaging sends the instant notifications to provider’s device.
5. JavaScript Logic handles the calculation while React Navigation moves us to Checkout screen.

1.5.5 Feature 5: Track Order (Customer App)

The Order Tracking and status feature, simplified and with technologies listed below:

1. After placing an order, user can check its current status at any time by simply tapping to “Orders” Section in the bottom navigation bar.
2. Request starts as “In Progress” but the moment provider accept the request, order status changes and now user can track the provider.
3. By tapping “Track on Map” button, user can exactly see where the provider is and watch them move towards user location.
4. After completion of service, user pay charges to the provider by hand.
5. Request status will be removed from the “Order” section.
6. Firebase Cloud Messaging handles the background notifications between two devices.

1.5.6 Feature 6: User Authentication (Provider App)

The list of features this module will have and how it will be used is as follow:

1. At first, providers select their specific skills such as “Sign-up as Mechanic” and “Sign-Up as Fuel Station”.
2. When provider opens the app, it automatically checks for an active session to see if provider is already logged in or not.
3. If verified or logged in already, they are sent to their dashboard; otherwise, they are guided to multi-step registration flow.
4. User Authorization have two options, “Sign-Up” or “Sign-In”.
5. First step of Sign-Up asks for basic identity details like name, email, Station/Workshop picture, address and password.
6. In second step, provider must enter their business details including CNIC, Uploading professional certificates, specializations, license Numbers and its expiry date.
7. Once the Sign-Up is submitted, the provider see the status “Application Under Review” on next page and cannot access home screen immediately.
8. The provider will access the dashboard once admin reviews and approve their upload documents.

9. Firebase authentication manages the login state and React Navigation handles the logic for switching between authentication and Home Screen.

1.5.7 Feature 7: Home Screen (Provider App)

1. Once logged in, providers are redirected to a specialized dashboard.
2. Bottom navigation bar gives provider quick access to their main workspaces, “Home”, “Orders”, “Profile”.
3. Providers registered as Mechanics can create a list of their services, such as Oil change, Brake Inspection and Tyre Burst, and set custom prices for each along descriptions.
4. Providers registered as Fuel Stations can list different fuel types like “High Octane”, “Diesel” and “Petrol” along with their current per-liter rates.
5. Providers can update their service charges based on changing market prices.
6. Firebase Firestore stores each service as a document with provider specific collection.
7. React Native forms allows for easy data entry.

1.5.8 Feature 8: Order Section (Provider App)

The Order Updates and tracking features for providers is follow as:

1. All requests sent by the customers appears in the Order Section.
2. Providers have flexibility to accept more than one request at a time.
3. When provider taps on “Accept” button, status changes on the customer order section and provider leaves for fulfilling the service.
4. Once the service is finished, provider taps “Completed” and after getting charges from customer, when this button is pressed, request will remove from active orders section.
5. OpenStreetMaps handle the live navigations, showing pinpoint location.
6. When provider taps on any button for updating status, notification will instantly go to the customer device.
7. All notifications will be handled by Firebase Cloud Messaging.

1.5.9 Feature 9: Profile Section (Customer & Provider App)

1. Profile section display all saved details such as name, email, profile picture etc.
2. End-users can edit their personal information.
3. A dedicated section is available to read through the app's privacy policy and terms of services to understand how data is protected.
4. Availability features allows provider to ON/OFF their availability by using simply toggle button.
5. Customer Feedback features is used to see all feedbacks for improving performance.
6. A logout button appears at the bottom, allows user to safely end their current session and exit the app.
7. Firebase authentication handles the secure sign-out process.

1.5.10 Feature 10: Admin Mobile App

The admin app will have the following features:

1. When admin successfully login, a dashboard will appears at the front screen, showing all statistics of Firebase Firestore.
2. Admin will see a list of options to select from, the options will be following:
    a. Pending Applications
    b. Customers List
    c. Manage Feedbacks
    d. Providers List
3. Admin can open the "Providers List" Section and have authority to view providers details, can view rating and their performance and also can suspend or delete their accounts based on it.
4. Admin can open "Customers List" Section and see all registered users, delete or suspend user accounts and also update their data.
5. Admin can open "Pending Applications" Section and review all service requests submitted by providers, examine all uploaded documents and verify them based on it.
6. Admin can view all service feedbacks and can analyze the provider performance.
7. All admin actions are connected with Firebase Firestore by fetching data in real-time and updates are quickly shown to the system.

1.6 Related System Analysis/Literature Review

Our Project primarily focuses on providing On-road assistance services. No other popular apps, websites and companies provide all types of services including breakdown services and fuel services in one platform. All these features in our app are free, our UI is easy to use and user can navigate throughout the app with ease. Our UI is not bulky, not too many features are stacked on one another on single screen. Most of these things are observed and we compare the differences of other platforms. A company rsapakistan.com provide on-road service but these are just limited to car repairing services and don't provide real-time locating feature. Also another app OkayKer provide just vehicle services but don't provide fuel service at a single platform. Our Platform provides all of these features on a single platform.

Table 1 Related System Analysis

Application Name	Weaknesses	Proposed Solution
Rsapakistan.com	Managed by a company; just limited to car repairing service only.	Connect users to a wide range of independent mechanics and fuel stations at reliable and user-friendly platform.
Hukumjanab.com	Provide both of these services but only available within Karachi and has limited geographical area.	Designed to work across multiple cities using OpenStreetMap.
OkayKer	Don't provide fuel delivery service and restricted to Lahore and Karachi.	Includes dedicated fuel delivery modules along mechanical repair module.

1.7 Vision Statement

Our vision is to develop a user-friendly mobile application that directly connects users to service providers for reliable services. Unlike traditional methods or limited service apps, QuickFix provides a smooth, all-in-one service platform for both mechanical breakdown

and fuel delivery and offering features like live location tracking. Platform aims to ensure convenience for customers to find services.

1.8 System Limitations and Constraints

The system will have the following limitations:

1. The system will not include online payment process.
2. System will not provide offline feature.
3. System will not provide any chat messaging feature.

The system will have the following constraints:

1. Our system will not support multi-lingua.
2. Platform must support 10,000 concurrent users.
3. System relies on manual Admin Approval.
4. Our system will not have web version.

1.9 Project Deliverables

Our system will have following deliverables by the end of 7th semester:

1. Project Proposal
2. Software Design Specification (SDS)
3. Prototype
4. Requirement Traceability Matrix (RTM)
5. Comprehensive Evaluation Document (Proposal, RTM, SDS)

Our system will have following deliverables by the end of 8th semester:

1. Coding/Implementation
2. Testing
3. Deployment
4. Final Documentation
5. Presentation & Final Evaluation

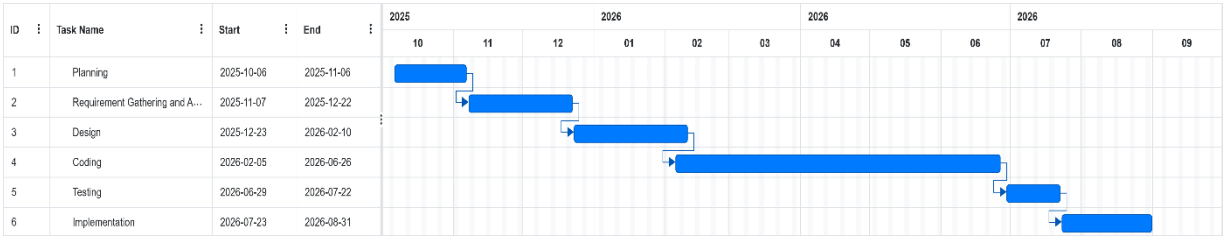
1.10 Tools and Technologies

Our system will use following tools and technologies for complete implementation of goals/objectives:

Table 2 Tools and Technologies

Tools/Technology/Library	Version	Rationale
Visual Studio Code	version: 1.104	Provides comprehensive set of features for coding.
React Native Maps and Geo-location service	version: 1.26.18	Used to display interactive maps and provide good access to device GPS.
Firebase Messaging (FCM)	version: 23.5.0	Enables to receive push notifications.
Firebase Authentication	version: 23.5.0	Handles user login and registration.
Firebase Storage	version: 23.5.0	Used to store Images.
Firebase Firestore	version: 23.5.0	Used to securely store and manage data.
Firebase Cloud Functions	version: 23.5.0	For backend processes.
React Native	version: 0.82.1	Build such an application that runs on both Android and IOS.
Firebase	version: 23.5.0	Scalable for server-less backend, manages FCM and authentication.
Open Street Maps	No version needed	Enables real-time tracking features.
Github	version: 1.252.0	To save a public repository for project submission.

1.11 Project Planning



ID	Task Name	Start	End
1	Planning	2025-10-06	2025-11-06
2	Requirement Gathering and A...	2025-11-07	2025-12-22
3	Design	2025-12-23	2026-02-10
4	Coding	2026-02-05	2026-06-26
5	Testing	2026-06-29	2026-07-22
6	Implementation	2026-07-23	2026-08-31

1.12 Summary

To get together with the emerging demand for digital roadside solution, we handle vehicle emergencies and ease the stress of finding reliable help on the road. Our project will deliver its modules and fulfil its goals within the planned timeframe. By maintaining a clear focus on the project scope, Quick aims to bridges the gap between stranded travelers and verified service providers.

Chapter 2

Requirements Analysis

2. Analysis

This chapter focuses on requirement identification, elicitation and analysis. The users categorized into four classes. For requirement identification and elicitation, open interviews, ethnography and use case analysis are used. Also, identified requirements are categorized into functional and non-functional requirements, along with their use case description and diagram.

2.1 User Classes and Characterization

The QuickFix will have four user classes, Customers, Mechanics, Fuel Stations and Admin.

Table 3 User Class and Characterization

User Classes	Characterization
Customer	The customer could be a driver or traveler who needs immediate help, such as mechanical repairs or fuel delivery.
Mechanic	Mechanic is a verified service provider who offers mechanical services and provide updates across on-going orders.
Fuel Seller	Fuel Station provides various fuel services and also updates their status in real-time.
Admin	The Admin is a specific individual with access to the management panel to verify provider documents and reviewing the entire QuickFix system.

2.2 Requirement Identifying Techniques

There are fundamental approaches used to identify requirements. As elicitation of requirements requires meeting with stakeholders so following are the approaches used:

2.2.1 Interviews

To truly understand the problems of being stuck on the road, we held open conversation with everyday travelers, drivers, workshop owners and fuel station staff. We analyze that on related platforms, when help was nearby, travelers and providers often struggled to find each other, which solidified our decision to build the open source platform which include all features along real-time tracking on map. Mostly platforms are limited to these features. These researches led us to build QuickFix as a flexible, accessible platform that can expand into any area.

2.2.2 Ethnography or Observation

Software systems do not exist in isolation. They are used in a social and organizational environment, and software system requirements may be generated or constrained by that environment. Ethnography is an observational technique that can be used to understand operational processes and help derive requirements for software to support these processes.

2.2.3 Use Case and Scenarios

Developing use cases and scenarios to describe how the system will be used in different situations. This technique helps in understanding the interactions between users and the system, making it easier to identify and document functional requirements. Use cases are a way of describing interactions between users and a system using a graphical model and structured text.

2.3 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. This section is organized by features as a system feature name and specific functional requirements associated with this feature. So, for each module and it's feature the functional requirements are itemized below:

2.3.1 Functional Requirement 1

Table 4 Functional Requirement 1

Identifier	FR-1
Title	Customer Sign Up

Requirement	The customer shall be able to create an account by entering basic info like name, email, address, password etc. then tapping the "Sign Up" button.
Source	Business Requirement
Rationale	Provide secure environment and collect essential profile data for personalization.
Business Rule	Email must be unique and valid along password must be at least 8 characters long and contains one special and numeric digit.
Dependencies	Internet Connection, Firebase Authentication, Firestore
Priority	High

2.3.2 Functional Requirement 2

Table 5 Functional Requirement 2

Identifier	FR-2
Title	Users Sign In
Requirement	Users shall be able to login securely using email and password.
Source	Business Requirement
Rationale	Provide secure environment.
Business Rule	Users must be registered before Sign In.
Dependencies	Internet Connection, Firebase Authentication
Priority	High

2.3.3 Functional Requirement 3

Table 6 Functional Requirement 3

Identifier	FR-03
Title	Password Reset
Requirement	The user shall be able to reset password by tapping a link on the login screen entering their email and receives a reset link.
Source	Business Requirement
Rationale	Reduce support costs by enabling self-service password recovery.
Business Rule	Password reset link must be expired after an hour.
Dependencies	FR-1
Priority	Medium

2.3.4 Functional Requirement 4

Table 7 Functional Requirement 4

Identifier	FR-04
Title	Search Nearby Providers
Requirement	Customers shall be able to search nearby providers.
Source	Business Requirement
Rationale	To find nearby services in emergency situations quickly.
Business Rule	GPS must be enabled
Dependencies	Location services
Priority	High

2.3.5 Functional Requirement 5

Table 8 Functional Requirement 5

Identifier	FR-05
Title	View Provider's List
Requirement	Customers shall be able to view a dedicated list of nearby service providers including mechanics and fuel stations.
Source	Business Requirement
Rationale	Help customers to find and pick the closest provider.
Business Rule	Show only nearest provider within 7km away.
Dependencies	Internet connection, Location service, Firebase Firestore
Priority	High

2.3.6 Functional Requirement 6

Table 9 Functional Requirement 6

Identifier	FR-06
Title	View Provider's Detail
Requirement	Customer can tap on a provider from the list to see their full profile and service details.
Source	Business Requirement
Rationale	Help users to verify providers or reviews before booking the service.
Business Rule	The "confirm order" button remains lock until the customer confirms what they want to request.
Dependencies	Internet Connection

Priority	High

2.3.7 Functional Requirement 7

Table 10 Functional Requirement 7

Identifier	FR-07
Title	Service Request
Requirement	Customer shall be able to create a service request after selecting required services and confirm.
Source	Technical Requirement
Rationale	To request help during vehicle breakdown or fuel shortage.
Business Rule	Location and type of services are required.
Dependencies	FR-06, Location Service
Priority	High

2.3.8 Functional Requirement 8

Table 11 Functional Requirement 8

Identifier	FR-08
Title	Track Order
Requirement	Customers shall be able to track the provider on map and find updates across their order.
Source	Business, Technical Requirement
Rationale	To provide better service experience
Business Rule	Map must refresh the provider's location every 10-15 secs.

Dependencies	FR-07, Location services, OpenStreetMaps
Priority	High

2.3.9 Functional Requirement 9

Table 12 Functional Requirement 9

Identifier	FR-09
Title	Multi-step Provider Sign Up
Requirement	The system shall guide providers through a multi-step registration including personal details, business details etc.
Source	Business Requirement
Rationale	To verify that every provider is trusted and qualified.
Business Rule	Account remains pending until admin approves their documents.
Dependencies	FR-01, Firebase Storage
Priority	High

2.3.10 Functional Requirement 10

Table 13 Functional Requirement 10

Identifier	FR-010
Title	Manage Services
Requirement	Providers shall be able to perform CRUD operations across services.
Source	Business Requirement
Rationale	To manage the requests of services effectively.

Business Rule	Every provider must keep one active service.
Dependencies	FR-09, FR-02, Internet Connection
Priority	Medium

2.3.11 Functional Requirement 11

Table 14 Functional Requirement 11

Identifier	FR-011
Title	Accept/Reject Request
Requirement	Providers shall be able to accept or reject service requests.
Source	Business Requirement
Rationale	To manage availability of providers.
Business Rule	If providers reject request, customers notified quickly so they can choose another one.
Dependencies	FR-07, Notification System
Priority	High

2.3.12 Functional Requirement 12

Table 15 Functional Requirement 12

Identifier	FR-012
Title	Update Request Status
Requirement	The provider shall be able to update service request status.
Source	Business, Technical Requirement

Rationale	To update customers about service request progress.
Business Rule	Status flow will be: Requested, Accepted, Completed
Dependencies	FR-011, Firestore, FCM
Priority	High

2.3.13 Functional Requirement 13

Table 16 Functional Requirement 13

Identifier	FR-013
Title	Profile Management
Requirement	Users shall be able to view their personal details and they shall be able to make changes across details.
Source	Business Requirement
Rationale	To keep user contact info current so that they can easily find and reach each other.
Business Rule	Changes must be verified. Incomplete profiles must be hidden and they will unable to send or receive requests.
Dependencies	FR-01, Firebase Storage
Priority	Medium

2.3.14 Functional Requirement 14

Table 17 Functional Requirement 14

Identifier	FR-014
Title	Notification System

Requirement	The system shall be able to send notifications across requests and updates.
Source	Business Requirement
Rationale	To provide information in real-time.
Business Rule	Notifications must be triggered on system events.
Dependencies	Internet Connection, FCM
Priority	High

2.3.15 Functional Requirement 15

Table 18 Functional Requirement 15

Identifier	FR-015
Title	Feedbacks
Requirement	Customer shall be able to submit rating and feedbacks and providers shall be able to view these feedbacks and rating.
Source	Business Requirement
Rationale	To improve the quality of the services.
Business Rule	Customer can only give feedbacks after the completion of service.
Dependencies	FR-07, Firebase Firestore
Priority	Medium

2.3.16 Functional Requirement 16

Table 19 Functional Requirement 16

Identifier	FR-016
Title	Manage Providers (Admin App)
Requirement	The admin shall be able to approve providers request based on documents and can suspend or delete their accounts.
Source	Business Requirement
Rationale	To ensure only verified providers all allowed to work.
Business Rule	Admin must approve or reject within 24 hours.
Dependencies	Firebase Firestore
Priority	High

2.3.17 Functional Requirement 17

Table 20 Functional Requirement 17

Identifier	FR-017
Title	Manage Customers (Admin App)
Requirement	Admin shall be able to view list of customers and have the ability to blocked their accounts.
Source	Business Requirement
Rationale	To monitor app usage and remove users who misbehave or create fake requests.
Business Rule	Blocked users cannot log in or create new requests.
Dependencies	Firebase Firestore

Priority	High

2.3.18 Functional Requirement 18

Table 21 Functional Requirement 18

Identifier	FR-018
Title	Pending Applications (Admin App)
Requirement	Admin shall be able to accept/reject all service requests.
Source	Business Requirement
Rationale	To track system performance and issues.
Business Rule	Admin can view all requests.
Dependencies	Firebase Firestore
Priority	High

2.3.19 Functional Requirement 19

Table 22 Functional Requirement 19

Identifier	FR-019
Title	View Feedbacks (Admin App)
Requirement	Admin shall be able to view all feedbacks and can take actions accordingly.
Source	Business Requirement
Rationale	To track service quality.
Business Rule	Admin can delete reviews that contains bad language or fake info.

Dependencies	Firebase Firestore
Priority	High

2.4 Non-Functional Requirements

These are constraints on the services or functions offered by the system. They include timing constraints, constraints on the development process, and constraints imposed by standards. Non-functional requirements often apply to the system as a whole rather than individual system features or services. Following are non-functional requirements:

Table 23 Non-Functional Requirements

Identifier	Category	Requirement Description	Module	Priority
NFR-1	Performance	The splash screen must load within 2 seconds to ensure user feels the app is responsive.	Module 1	High
NFR-2	Performance	Map data and nearby provider lists should load within 5 seconds on 3G/4G connection.	Module 2	High
NFR-3	Scalability	System must handle up to 10000 concurrent users and 5000 active requests.	All modules	Medium
NFR-4	Usability	UI must follow material design guidelines.	All modules	High
NFR-5	Responsiveness	Layout will be fully responsive, adjust seamlessly for all Android and IOS screen	All modules	High
NFR-6	Data Retention	All service requests and history upon orders must be stored for at least one year.	Module 2,4,5	Medium

2.5 Use Case Analysis

All systems involve interaction of some kind. This can be user interaction, which involves user inputs and outputs, interaction between the software being developed and other systems in its environment, or interaction between the components of a software system. User interaction modeling is important as it helps to identify user requirements.

2.5.1 Use Case 1

Table 24 Use Case 1

Use Case ID	UC-1
Use Case Name	Customer Sign Up
Actors	Customers
Description	This use case allows a new customer to register by providing their personal details and credentials.
Trigger	Customer selects the “Sign Up” option from the Authorization screen.
Pre-conditions	Customer must not already be registered.
Post-conditions	A new customer account is created in Firebase, and the customer is redirected to the Customer Home screen.
Normal Flow	- Customer opens the app. - Authorization screen appears. - Customer selects “Sign Up”. - Customer enter Name, Email, Address, Picture and Password. - Form is validated. - Firebase authentication creates user. - User is redirected to Home Screen.
Alternative Flows	- Email Already Exists: If the email is already registered, a message is shown to user.

	- Weak Password: If password does not meet criteria, user is prompted to correct it.
Business Rules	- Email must be unique and valid. - Password must be at least 8 characters, with one special letter and one numeric letter. - Required fields cannot be empty.
Assumptions	- Firebase Authentication is active and integrated. - User device has internet access.

2.5.2 Use Case 2

Table 25 Use Case 2

Use Case ID	UC-2
Use Case Name	Login with Email and Password
Actors	End User
Description	This use case allows a registered user to login to the application using their email and password.
Trigger	The user opens the app and select the “Login” option on Authorization screen.
Pre-conditions	The user must already be registered with a valid email and password.
Post-conditions	If the login is successful, the user is redirected to the home screen. If not, an error message is shown.
Normal Flow	- User opens the app. - Splash screen checks login status. - User is not authenticated and is taken to the Authorization screen. - User selects “Already have an Account, Login”. - User enter email and password.

	- App validates and verifies credentials with Firebase Authentication. - If valid, user is redirected to the home screen.
Alternative Flows	- Invalid Credentials: If the entered email or password is incorrect, the user is shown an error message and remains on the login screen. - Unverified User: If user email is not verified, user cannot login. - Forgot Password: User selects "Forgot Password" enters registered email, receives a reset email, and follows the link to reset the password.
Business Rules	- Email and password must be filled and validated. - Email must be registered. - Email must be verified. - Password must meet Firebase Authentication requirements.
Assumptions	- The user has stable internet connection (4G/5G). - Firebase services are available. - The app is correctly configured to use Firebase Authentication.

2.5.3 Use Case 3

Table 26 Use Case 3

Use Case ID	UC-3
Use Case Name	Forgot Password
Actors	End User
Description	This use case allows a user to reset their password via email.
Trigger	User selects the "Forgot Password" link on the login screen.
Pre-conditions	User must have a registered email address.
Post-conditions	Password reset link is sent to the user's email.

Normal Flow	- User clicks “Forgot Password”. - User enters registered email address. - App sends reset link via Firebase. - User follows link to reset password.
Alternative Flows	- Email Not Registered: User enters unregistered email show error. - Email Sending Failed: Network issue or Firebase error show message.
Business Rules	- Email field must be valid and not empty. - Firebase handles email sending and validation.
Assumptions	- Firebase Authentication is functional. - User checks correct inbox/spam folder.

2.5.4 Use Case 4

Table 27 Use Case 4

Use Case ID	UC-4
Use Case Name	Search Nearby Providers
Actors	Customers
Description	This use case allows a customer to find available mechanics within their current geographic area (within 7 km).
Trigger	User selects the “Search bar” on the top of home screen.
Pre-conditions	Location service must be enabled on the user’s device.
Post-conditions	A list of active and approved providers is displayed to the customer.
Normal Flow	- Customer opens the app. - After opening app, the main home screen appears. - Customer accesses the search bar option.

	- System fetches current GPS coordinates. - System goes to Firebase for providers within customer radius. - System filters for “Active” and “Approved” status. - Providers are displayed on another screen.
Alternative Flows	- No Providers Found: System displays a “No providers in your area” message. - Location Denied: System prompt customer to enable GPS settings manually.
Business Rules	- Only providers with at least one active service are shown. - Only “Approved” providers are visible. - Search radius is limited to 7km by default.
Assumptions	- OpenStreetMap API is integrated. - Device has a functional GPS sensor.

2.5.5 Use Case 5

Table 28 Use Case 5

Use Case ID	UC-5
Use Case Name	View Provider’s List and Details
Actors	Customer
Description	This use case allows customer to browse list of nearby providers and view their specific services with details.
Trigger	Customer selects the “View Details” or tap provider icon on the screen.
Pre-conditions	Nearby providers must have been successfully retrieved.
Post-conditions	Customer view the full profile and services with price and description of provider.

Normal Flow	- System displays a list of providers with name, rating and distance. - Customer selects a specific provider. - System fetches and displays provider's service list and pricing. - Customer reviews details to decide on a service request.
Alternative Flows	- No Services Listed: If a provider has no services they are removed from the list. - Back to List: Customer toggles back to list view.
Business Rules	- Providers are sorted by proximity. - Prices must be displayed in the local currency. - Ratings are shown as an average of all previous feedbacks.
Assumptions	- UI follows Material Design list item standards. - Data is pulled from the "Services" sub-collection in Firebase.

2.5.6 Use Case 6

Table 29 Use Case 6

Use Case ID	UC-6
Use Case Name	Service Selection
Actors	Customer
Description	This use case allows customer to select one or more services from the provider's menu.
Trigger	Customer taps on checkbox to select services from the list.
Pre-conditions	Customer is logged in and has selected a nearby service provider.
Post-conditions	Selected services are added to the "Checkout Summary" and user is directed to the checkout phase.
Normal Flow	- System displays the provider's available service listing.

	- Customer select desired services via checkboxes. - System calculates the subtotal of selected services. - Customer taps “Next” to proceed.
Alternative Flows	- No selection: If customer taps “Next” without selecting service, system shows a “Please select a service” popup.
Business Rules	- At least one service must be selected to proceed. - The system must shows the base price for each service clearly.
Assumptions	- Service data is fetched in real-time from the provider’s database record. - UI uses a selection list (Checkboxes).

2.5.7 Use Case 7

Table 30 Use Case 7

Use Case ID	UC-7
Use Case Name	Checkout Process
Actors	Customer
Description	This use case allows customer to review their selected services, current location and then send the final request to the provider.
Trigger	Customer taps “Confirm Order” after reviewing services.
Pre-conditions	Services must be selected and GPS location must be active.
Post-conditions	A service request document is created in Firebase and a notification is sent to the provider.
Normal Flow	- System displays a summary of selected services and total cost. - Customer reviews all selected services. - Customer adds an optional description of the problem.

	- Customer confirms the current location. - Customer taps “Confirm Order”.
Alternative Flows	- Missing details: System highlights required fields if left empty. - Cancellation: Customer exits checkout, and the temporary selection is cleared.
Business Rules	- GPS location must be on during request order. - The customer cannot send a second request while the one is pending.
Assumptions	- Internet connectivity is active. - The provider is currently “Online” and able to receive push notifications.

2.5.8 Use Case 8

Table 31 Use Case 8

Use Case ID	UC-8
Use Case Name	Track Order
Actors	Customer
Description	This use case allows customer to monitor the status of their request and allows to track provider’s location after the request is accepted.
Trigger	Provider accepts the service request.
Pre-conditions	Customer has submitted a service request and the provider has accepted it.
Post-conditions	Customer can track the provider’s current location while the accepted request is in progress..
Normal Flow	- System notifies customer that the request has been accepted. - Customer taps on ‘Track On Map” section.

	- System displays a live map showing the provider's current GPS position. - System continuously notifies customer that provider is on the way, and arrived using marker directing towards customer. - System shows estimated time of arrival. - Provider taps "Order Completed" after request is completed.
Alternative Flows	- If the provider's current location is unavailable, the system informs the customer that the provider location is not available yet. - Cancellation: If the provider cancels the request, customer notifies instantly.
Business Rules	- Location is updated periodically while location tracking is active. - Tracking auto ends once the provider marked as "Order Completed".
Assumptions	- Internet connectivity is available for retrieving location data and displaying the map

2.5.9 Use Case 9

Table 32 Use Case 9

Use Case ID	UC-9
Use Case Name	Provider Sign Up
Actors	Service Provider
Description	This use case allows provider to create an account by providing personal details and business details, uploading verifications documents for admin review.
Trigger	Provider selects "Sign up as a Mechanic" or "Sign up as a Fuel station" from the authorization screen.

Pre-conditions	Provider must not have an existing provider account associated with their details.
Post-conditions	A provider document is created is Firebase with a “Pending” status.
Normal Flow	- Provider select their specific role from authorization screen. - Provider enters their personal details including name, email, address, and password. - Provider enter their business details and upload required documents for verification. - Provider submits the application for review.
Alternative Flows	- Invalid document Format: System displays an error if the file is not a supported image. - Incomplete Profile: System prevents submission if mandatory fields are empty.
Business Rules	- Email must be unique and verified. - Account remains “Pending” and home screen will not show to provider until admin approval. - All uploaded documents must be clear and legible.
Assumptions	- Provider has a stable network connection to upload documents. - Firebase Authentication is properly configured.

2.5.10 Use Case 10

Table 33 Use Case 10

Use Case ID	UC-10
Use Case Name	Manage Services
Actors	Service Providers

Description	This use case allows provider to add, view and edit their offered services directly from their Home Screen.
Trigger	Provider taps the “Add Service” button on the home screen.
Pre-conditions	Provider is logged in and their account status is approved.
Post-conditions	Service updates are saved in Firestore and immediately visible to customers.
Normal Flow	- Provider views the “+/Add” section on the Home Screen. - Provider selects “Add Service”. - Provider enters Service Name (Oil Change). - Provider enters the description. - Provider sets the price (e.g. $20). - Provider selects “Save”.
Alternative Flows	- Missing Fields: If price or title is empty, the “Save” button remains disabled. - Invalid Prices: If a negative number or text is entered in the price field, an error will be shown.
Business Rules	- All services must have a non-zero price. - Descriptions are limited to 200 characters for UI consistency. - Updates reflect in real-time on the customer side.
Assumptions	- The Home Screen UI is reactive. - Firestore document structure supports a “service” sub-collection for each provider.

2.5.11 Use Case 11

Table 34 Use Case 11

Use Case ID	UC-11

Use Case Name	Service Updates
Actors	Service Providers
Description	This use case allow providers to view incoming request, accept or reject them and mark an accepted request as completed after providing the service.
Trigger	Provider receives a push notification for new request.
Pre-conditions	Provider status must be online and account must be approved.
Post-conditions	The service request status is updated in Firebase according to the provider's action, such as Accepted, Rejected, or Completed.
Normal Flow	- Provider receives a new service request in "Order" section. - Provider views the requested service details and taps "Accept". - System changes the status from Pending to Accepted in Firestore. - Customer can use "Track On Map" for the accepted request. - Provider taps "Completed" after doing work and receive payment.
Alternative Flows	- Reject Request: Provider taps "Rejected", push notification will send to the customer.
Business Rules	- Every status change must trigger a notification to the customer.
Assumptions	- Firebase Cloud Messaging is active for real-time notifications. - Provider has background location permissions enabled.

2.5.12 Use Case 12

Table 35 Use Case 12

Use Case ID	UC-12
Use Case Name	Manage Profile
Actors	Customers, Providers

Description	This use case allows user to view and update their personal details.
Trigger	User selects the “Profile” section icon from the navigation menu.
Pre-conditions	User must be logged into the application.
Post-conditions	Updated profile data is synchronized with Firebase Firestore and Authentication.
Normal Flow	- User navigates to the Profile screen. - System displays current details. - User taps “Edit Profile”. - User modifies address, photo etc. - User taps “Save”. - System validates data and updates the database.
Alternative Flows	- Invalid Address: System shows an error if the phone format is incorrect. - Upload Error: If the photo file size is too large, system prompts for a smaller image.
Business Rules	- Email address cannot be changed once the account is verified. - Profile photo must be clear. - User cannot save a profile if mandatory fields are left empty.
Assumptions	- User has granted camera permissions. - Firestore is used to store the user’s profile document.

2.5.13 Use Case 13

Table 36 Use Case 13

Use Case ID	UC-13
Use Case Name	Providers List
Actors	Admin

Description	This use case allows admin to review provider registration applications and view registered provider details, and disable or permanently delete provider accounts.
Trigger	Admin logs into the Admin Panel and selects “Providers List”.
Pre-conditions	Admin must be authenticated and authorized to access the Admin Panel.
Post-conditions	Provider account status is updated.
Normal Flow	- Admin opens the Providers List. - System displays the registered service providers. - Admin reviews the provider information. - Admin may return to the Providers List or perform an available account management action.
Alternative Flows	- Disable Account**:** Admin selects “Disable Account” and confirms the action. - Delete Account: Admin selects “Delete Permanently” and confirms the action.
Business Rules	- Permanent deletion removes the provider record from Firestore. - Disabling an account changes the provider’s status to disabled.
Assumptions	- Admin Panel is synced with Firebase Firestore for real-time status updates. - Admin is authenticated and authorized to access provider management features.

2.5.14 Use Case 14

Table 37 Use Case14

Use Case ID	UC-14

Use Case Name	Customers List
Actors	Admin
Description	This use case allows admin to view and manage registered customers. The admin can view customer details, disable a customer account, or permanently delete the customer record.
Trigger	Admin opens the Customers List from the Admin Panel.
Pre-conditions	Admin must be logged in and have access to the Admin Panel.
Post-conditions	Customer information remains unchanged if only viewed, or the customer account status is updated according to the admin's action.
Normal Flow	- Admin opens the Customers List. - System displays the list of registered customers. - Admin selects a customer from the list and the system displays the selected customer's details. - Admin reviews the customer information. - Admin may return to the Customers List or perform an available account management action.
Alternative Flows	- Disable Account: Admin selects Disable Account and confirms the action. System changes the customer account status to disabled. - Delete Account: Admin selects Delete Permanently and confirms the action. System removes the customer record from Firestore.
Business Rules	- Disabling an account changes the customer's status to disabled**.** - Permanent deletion removes the customer record from Firestore.
Assumptions	- Admin is authenticated and authorized to access customer management features. - Firebase Firestore is available for retrieving and updating customer records.

2.5.15 Use Case 15

Table 38 Use Case 15

Use Case ID	UC-15
Use Case Name	Customer Feedbacks
Actors	Admin
Description	This use case allows admin to view ratings and feedback submitted by customers for providers after completion of a service request.
Trigger	Admin opens the Customer Feedbacks section from the Admin Panel.
Pre-conditions	At least one customer feedback record has been submitted and stored in the system.
Post-conditions	Feedback is either maintain or deleted from the profile.
Normal Flow	- Admin opens the Feedback section. - System retrieves submitted feedback records from Firestore. - System displays the available feedback records. - Admin views the feedback information and take actions accordingly.
Alternative Flows	- No Feedback Available: If no feedback has been submitted, the system displays an empty feedback message.
Business Rules	- Each feedback record is associated with the relevant customer and provider. - Admin has view-only access to feedback through this use case.
Assumptions	- Feedback records contain the customer and provider information required to identify the related parties.

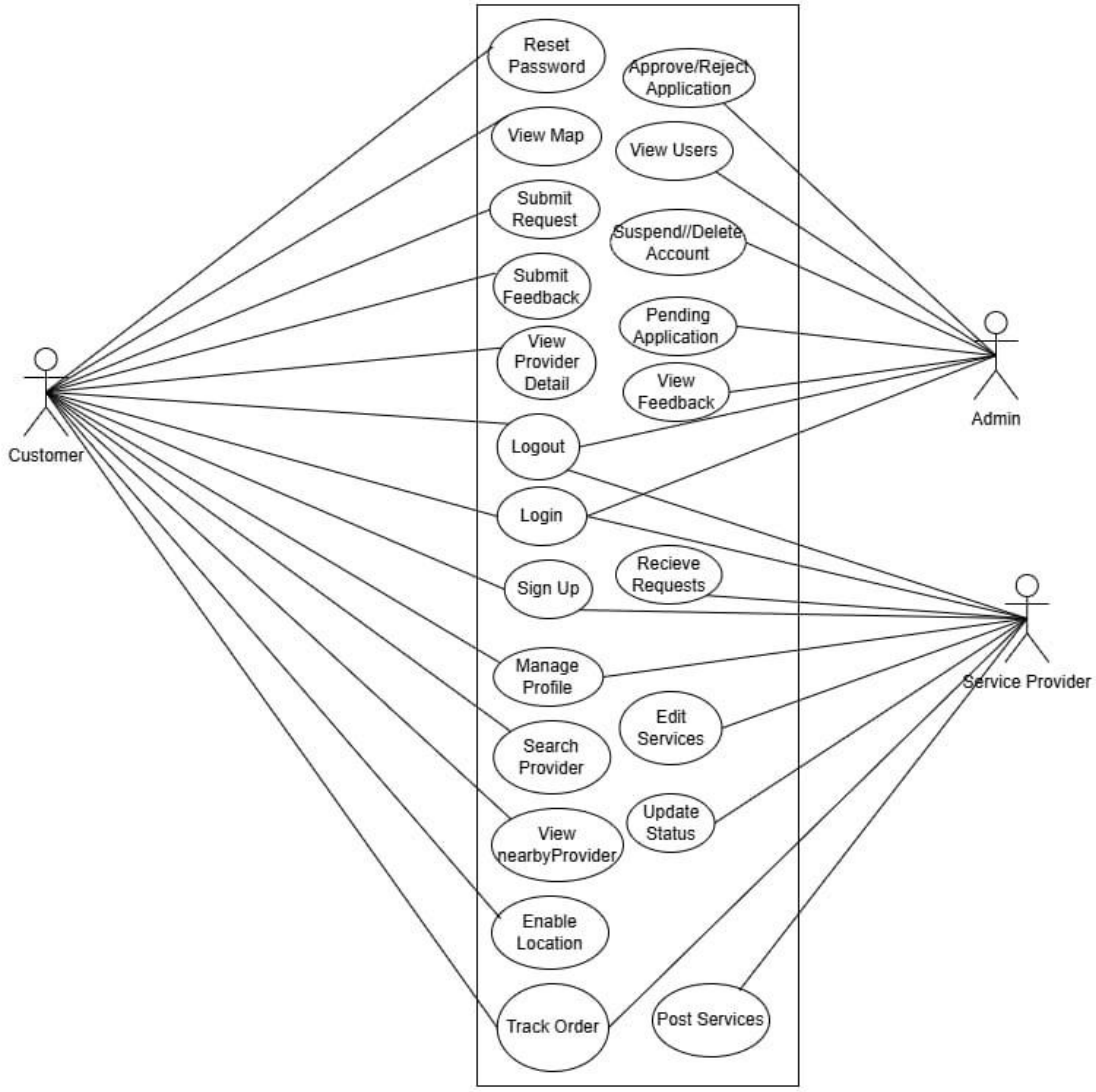
2.5.16 Use Case 16

Table 39 Use Case 16

Use Case ID	UC-16
Use Case Name	Pending Applications
Actors	Admin
Description	This use case allows admin to view and review pending applications submitted by Mechanics and Fuel Station Providers and approve or reject them after reviewing their submitted information and verification documents.
Trigger	Admin opens the “Pending Applications” section from Admin Panel.
Pre-conditions	Admin must be logged in and at least one provider application with pending status must exist in the system.
Post-conditions	The selected provider application is either approved or rejected, and its status is updated in Firestore.
Normal Flow	- Admin opens the Pending Applications section. - Admin reviews the submitted verification documents. - Admin selects accept and the ystem updates the provider’s status from pending to approved**.**
Alternative Flows	- Reject Application: Admin reviews the application and selects Reject**.** The system updates the provider application status accordingly.
Business Rules	- Only applications with pending status are displayed in the Pending Applications section.
Assumptions	- Required provider information and verification documents are available for admin review.

2.5.17 Use Case Diagram

Figure 1 Use Case Diagram



2.6 External Interface Components

This section describes how QuickFix interacts with users, external software components, hardware, and communication channels.

2.6.1 User Interface Requirements

The QuickFix app interface shall adhere to the following design and usability standards:

Table 40 User Interface Components

Category	Requirements

General Design	The application shall maintain a clean and consistent layout across all screens, with readable text, consistent colors and controls.
Navigation	A persistent bottom navigation for every Customer and Provider with options like Home, Orders and Profile. The Admin interface shall provide navigation to administrative sections.
Dashboard Layout	Customer dashboard includes map view, search bar and nearby Providers. Provider dashboard shows all service listings and incoming requests.
Forms	All forms (registration, submitting request) shall display required field indicators (*) and validation messages.
Buttons and Controls	Standard buttons such as request service, accept, reject, track and submit feedback must be consistently designed.
Screen Resolution	Fully functional on mobile screens including Android and iOS devices.
Loading Indicator	Loading indicators shall be displayed where the application is waiting for data retrieval or other processes when required.
Success Message	The system shall display a clear confirmation message after successful operations.
Error Message	The system shall display clear error or validation messages when an operation fails or invalid information is provided.
Map Integration	Interactive map must be displayed for location tracking and Providers navigation.
Accessibility	Text, icons, labels, and visual indicators shall be presented clearly and consistently to make application fully functional to users.

2.6.2 Software Interfaces

The following software components are interfaced by QuickFix:

Table 41 Software Interfaces

Component	Interface Description
Firebase Authentication	Secure user authentication with email/password and account management.
Firebase Firestore	Stores and manages application data including customer/provider profiles, provider services, service requests, request statuses, and feedback.
Firebase Storage	Stores uploaded images such as profile pictures.
Firebase Cloud Messaging	Send real-time notifications to Customers and Providers.
Device Location Services / Geolocation	Provides access to the device's coordinates for customer and provider location-based functionality and real-time provider tracking.
React Native Framework	Provides the framework used to develop the mobile application using a shared JavaScript-based codebase.
OpenStreetMap with Leaflet	Provides the interactive map displayed in the tracking interface for showing customer and provider locations.

2.6.3 Hardware Interfaces

QuickFix is a purely mobile platform. It does not require any dedicated external hardware peripherals.

Table 42 Hardware Interfaces

Components	Description
Smart Phones	Primary device used by Customers and Providers to access the application.
Location Services	Used for real-time location tracking and find nearby service.

Touch Screen Interface	Used for navigation, booking services and interaction with application.
Network Interface	Provides network connectivity required for communication with Firebase services.

2.6.4 Communication Interfaces

The following communication protocols and services are used by QuickFix:

Table 43 Communication Interfaces

Protocol/ Service	Description
HTTPS	Secure network communication is used when the application communicates with Firebase services and other online resources.
Firestore Real-Time Communication	Firestore real-time listeners are used to receive updated application data, such as service request and order status changes, without requiring manual refresh.
TCP/ IP	Standard Internet networking protocols are used for communication between the mobile application and online services.
Secure Port (HTTPS)	HTTPS-based communication uses standard secure network communication, typically through port 443.

2.7 Summary

This chapter outlines the process of identifying, eliciting and analyzing system requirements. Users are divided into two main categories. To gather requirements, techniques such as open interviews, ethnographic observation and use case analysis are applied. The collected requirements are then classified into functional and non-functional types, supported by detailed use case descriptions and diagrams.

Chapter 3

Design and Architecture

3. System Design

Product Perspective

QuickFix system is a multi-sided service platform designed to quickly connect people with nearby service providers. System ensures smooth experience by providing cross-platform access to both Android and IOS. It provides core features such as instant service providing, continuous live status tracking and feedback system. Application follows the Model-View-Presenter architectural pattern to ensure scalability and maintainability. This architecture make sure everything runs smoothly by keeping business logic separate inside scalable services on backend.

Dependencies & Interactions

- Authentication relies on Firebase authentication service for secure login.
- Firebase cloud messaging send push notifications to users based on events.
- Firebase backend execute backend logic also triggers on database events and handle notifications.
- To access features like GPS tracking, mobile app rely on device's OS.

Design Constraints

- All crucial tasks across application must be completed in under 3 seconds.
- Platform must Support 10,000 concurrent users.
- System will not support multi-lingua.
- App interface must be responsive, ensure easy use by following native standards.
- Follows MVP pattern so we can easily update parts of the system.

3.1 Design Considerations

3.1.1 Assumptions and Dependencies

- Both customers and providers have smart phone and have reliable internet Connection (4G/5G).
- GPS location must be enable across all devices.
- System relies on firebase authentication for secure login.
- FCM Gateway for instant notifications delivery.
- Data written to firestore assumed to be structurally correct.

3.1.2 Limitations

- System will not include online payment process.
- System will not provide offline feature.
- System will not include chat messaging.

3.1.3 Risks and Obstacles

Table 44 Risks and Obstacles

Risk & Obstacle	Impact	Mitigation Strategy
Firebase service outages effect core functionalities.	User and provider face problems during service execution.	Use local data storage to allow limited offline features.
Mechanic arrives without the correct tools for breakdown.	In this way, people can get stressful and wasting their time.	We will let the user to upload the picture of specific problem.
Risk of data breaches.	Sensitive information of users may be misused.	Use firebase security features.

3.2 Design Models

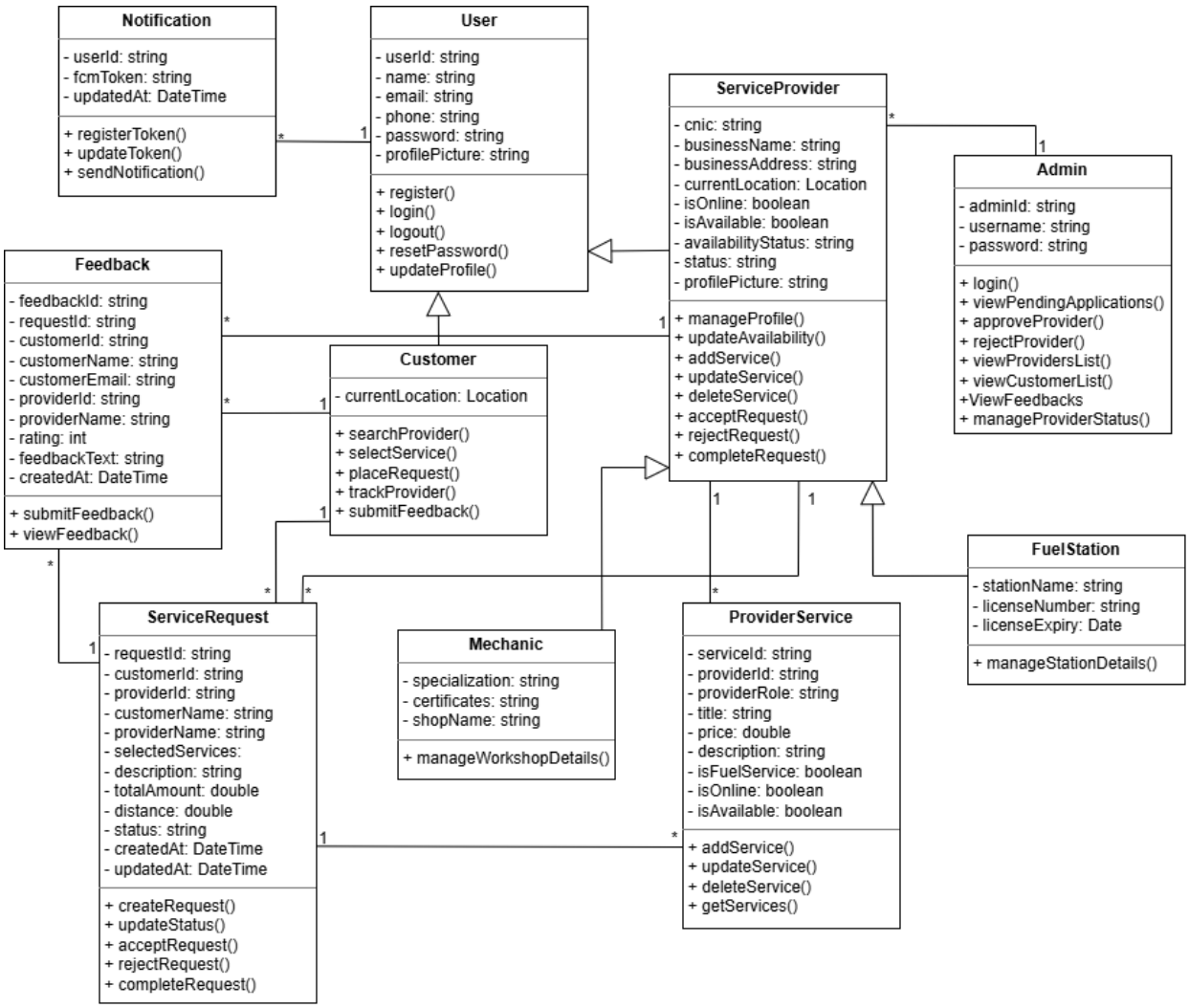
The system design model acts as an essential blueprint for persistent information within the application. It is the architectural standard that defines the structure and relationships of data before any code is written. Beside a full representation of the system it's a strategic abstraction that maps the core business entities to technical resources and functionalities. This model is crucial because it ensures that everyone in the team agrees on exact structure of the data. It prevent errors and make sure all information remains consistent throughout the entire QuickFix system.

3.2.1 Class Diagram

The class Diagram models the static structure of the software. It defines the core components required for the system to function. It clearly define each class by detailing its internal attributes and functions it perform. This model illustrates various connections and

dependencies between these classes. Its purpose is to provide a clear map of system structure which shows exactly how all the data and functions are grouped and connected.

Figure 2 Class Diagram



3.2.2 Sequence Diagram

The sequence diagram models the dynamic behavior of the system. It defines the order of events over time. It shows the involved entities and the exchange of messages or interactions which are necessary to complete a particular operation. By placing these events, diagram ensure the precise order and timing of all steps. It ensures that every action happens in the correct and predetermined sequence. Here we will make sequence diagram of every step follow as:

Figure 3 Sequence Diagram 1

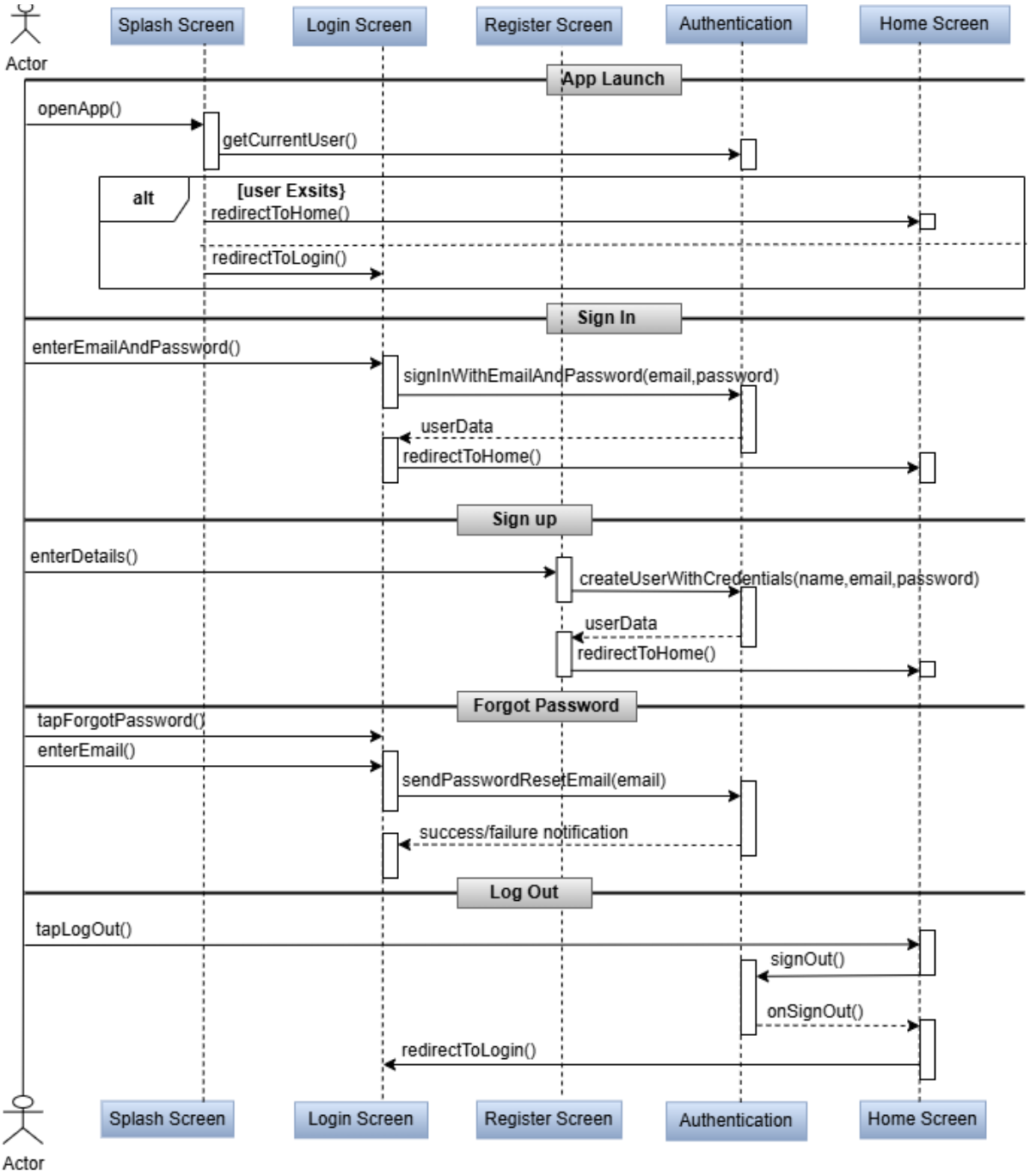


Figure 4 Sequence Diagram 2

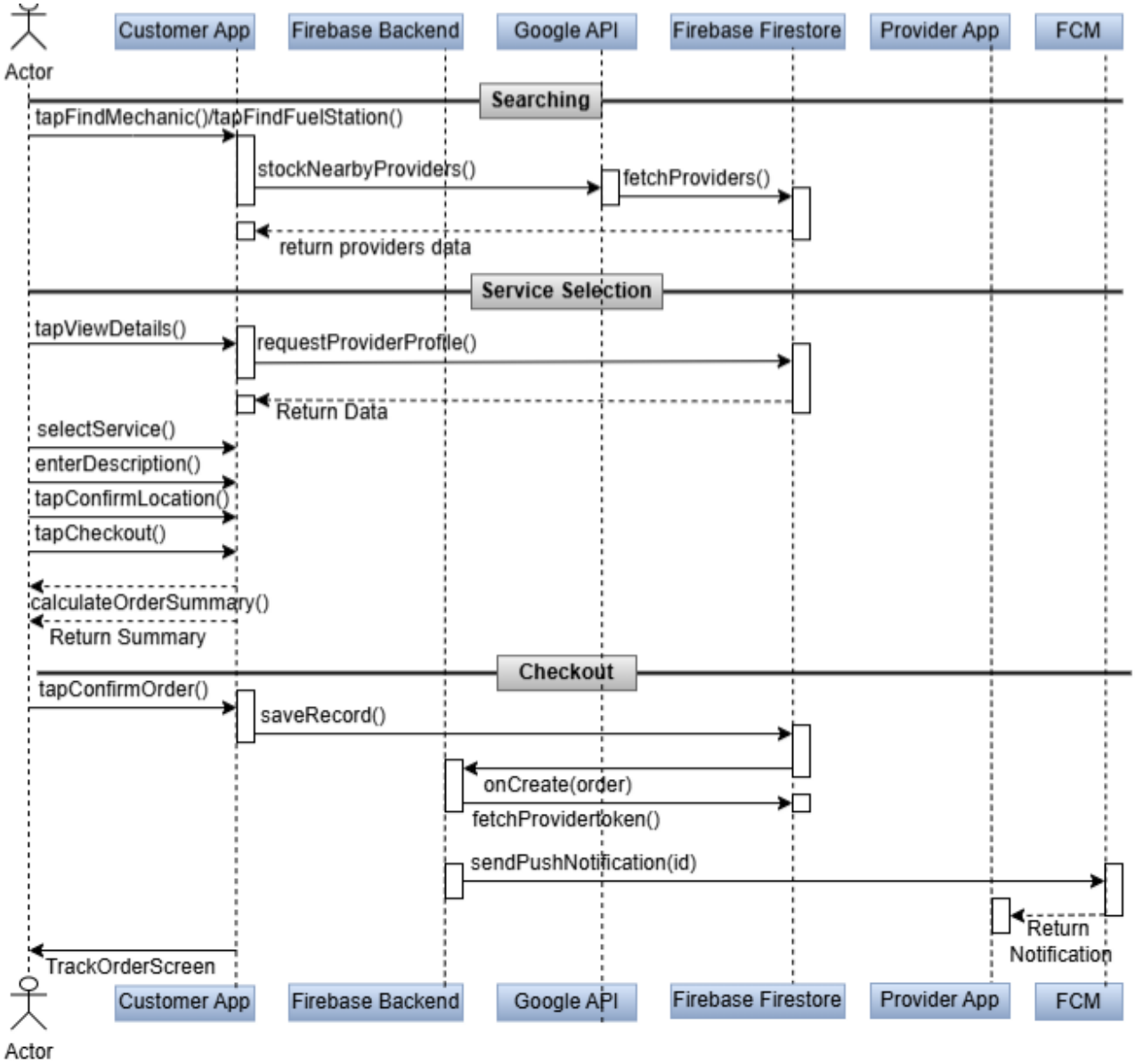


Figure 5 Sequence Diagram 3

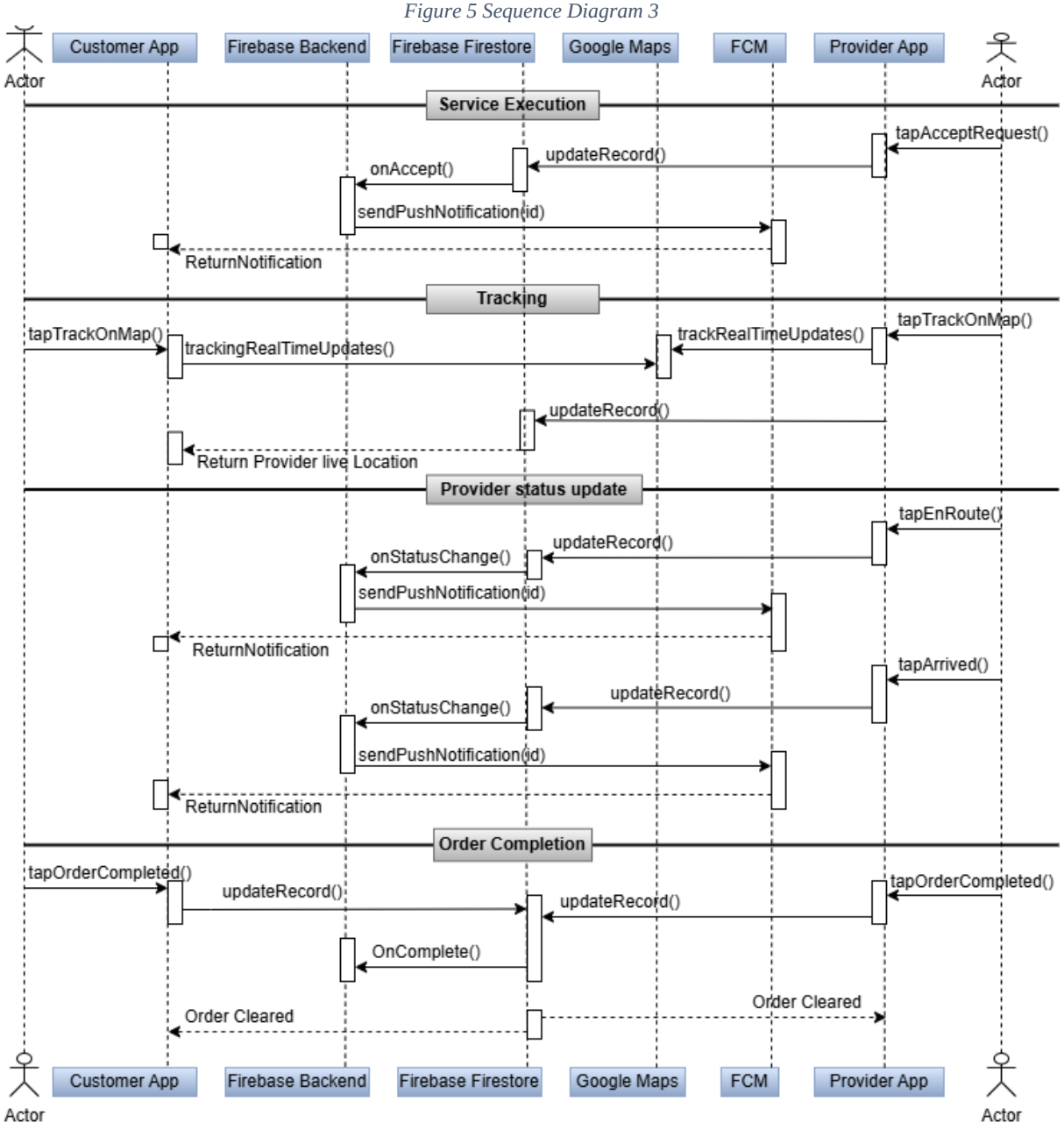


Figure 6 Sequence Diagram 4

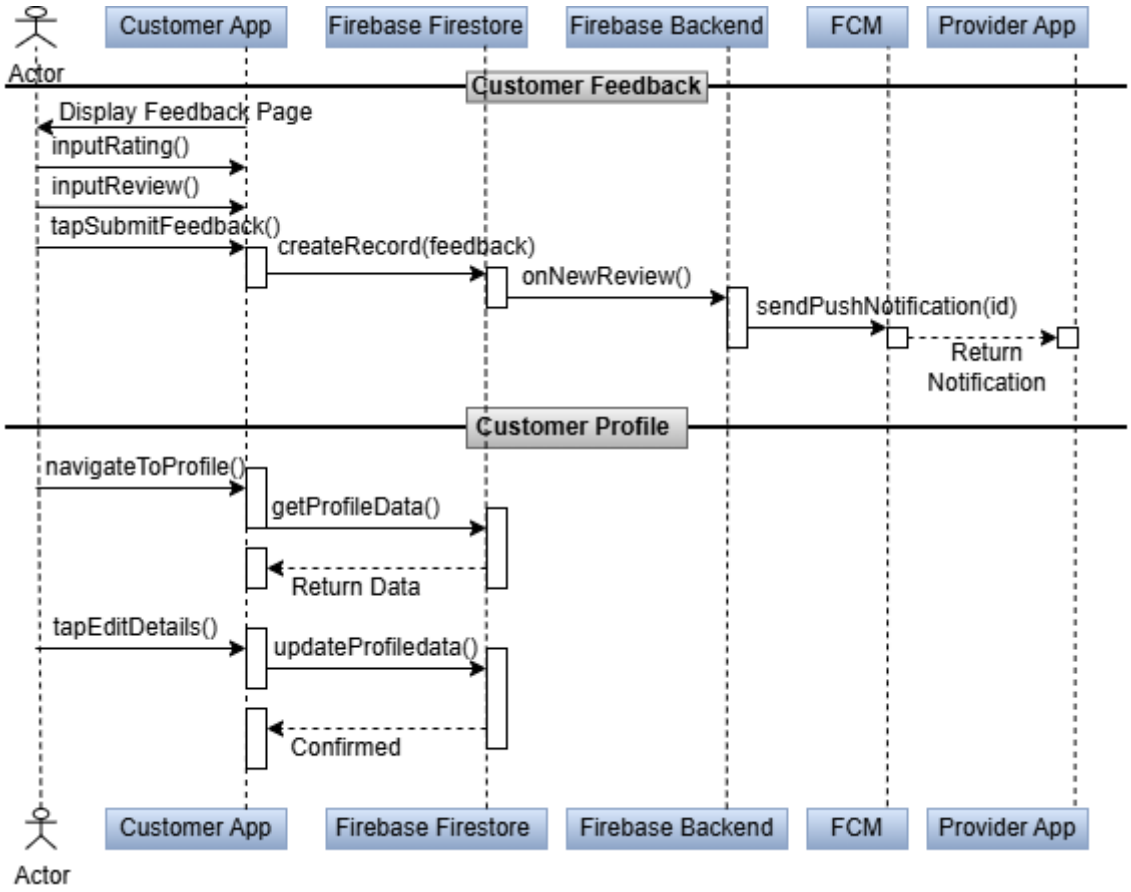


Figure 7 Sequence Diagram 5

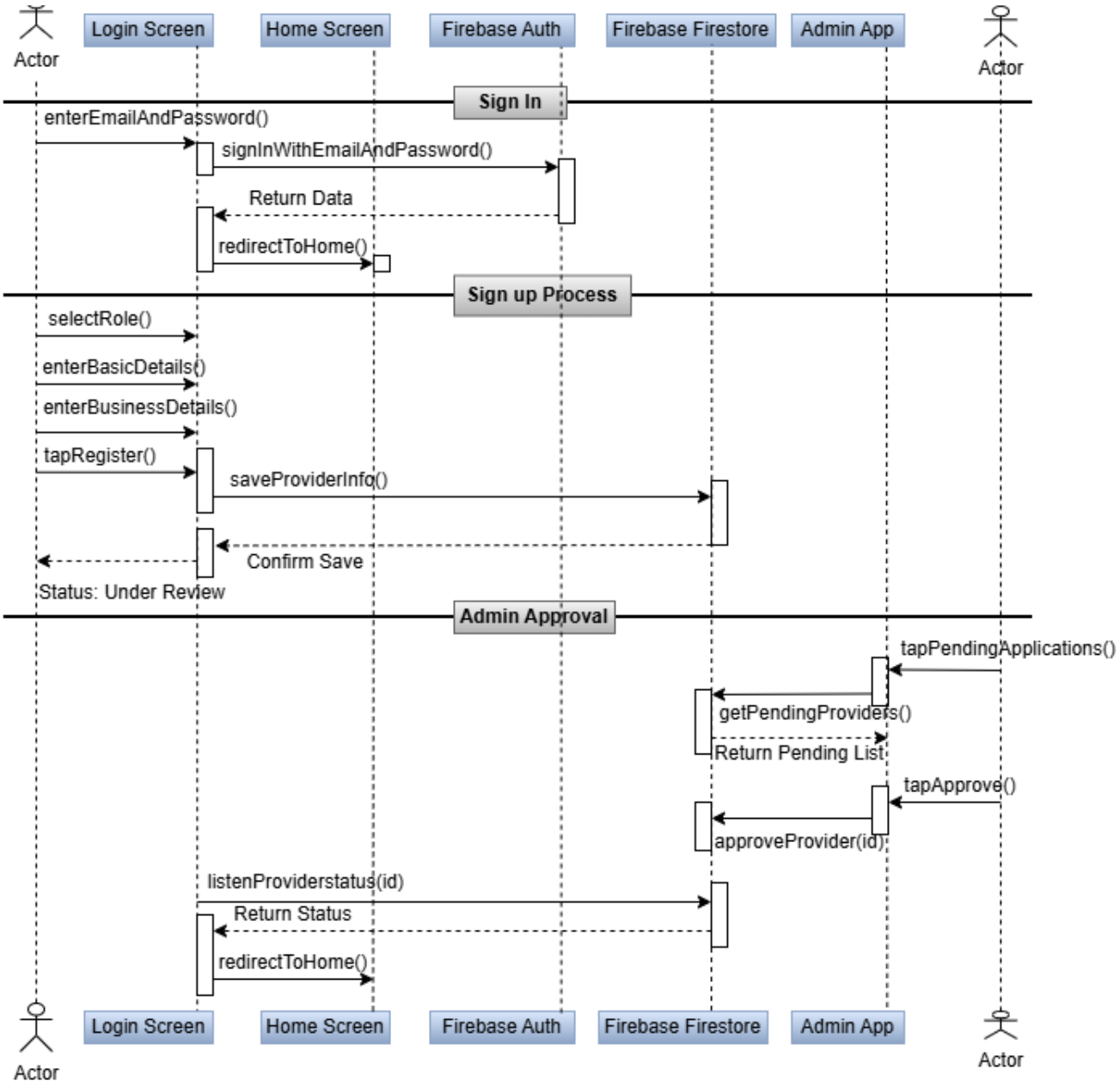


Figure 8 Sequence Diagram 6

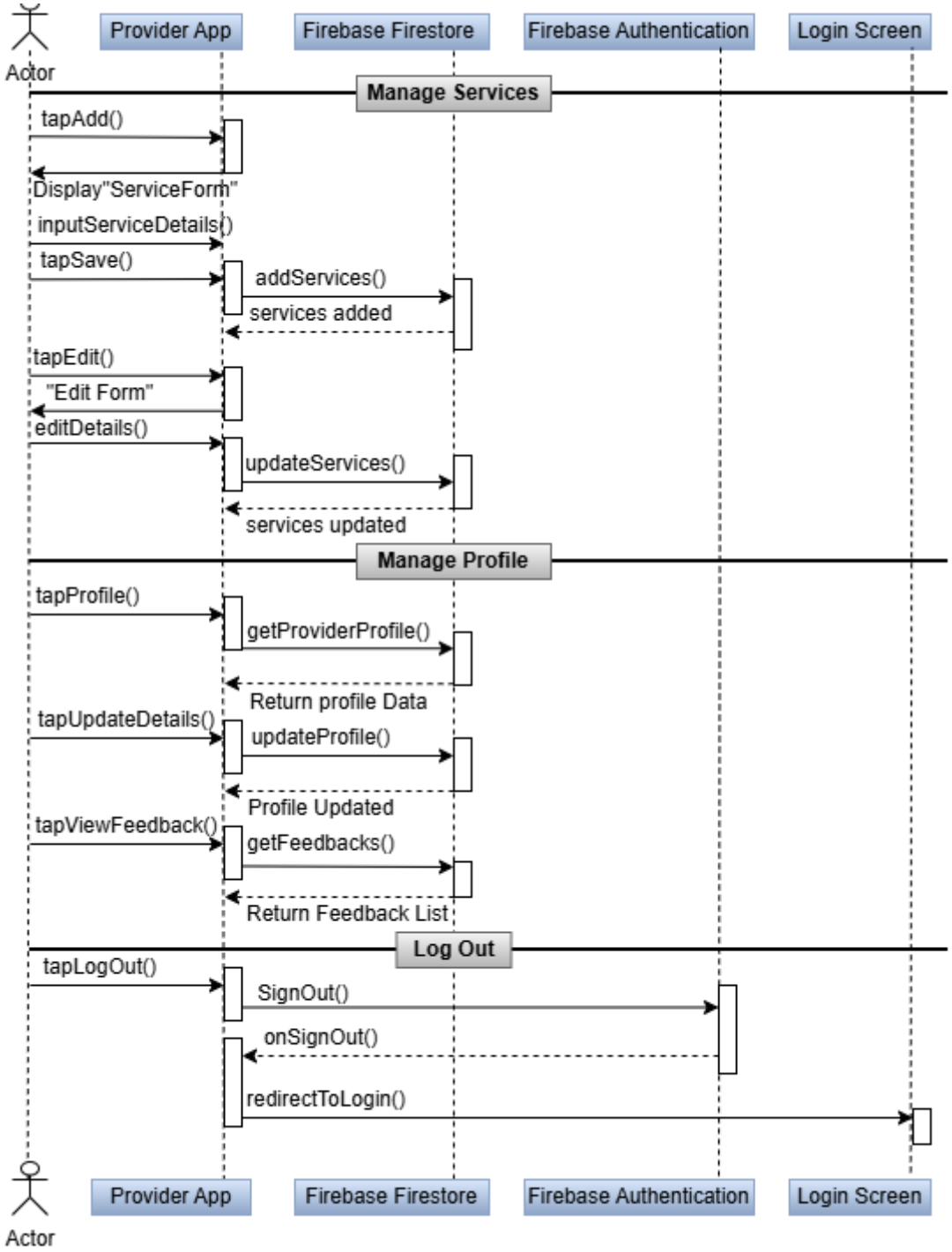
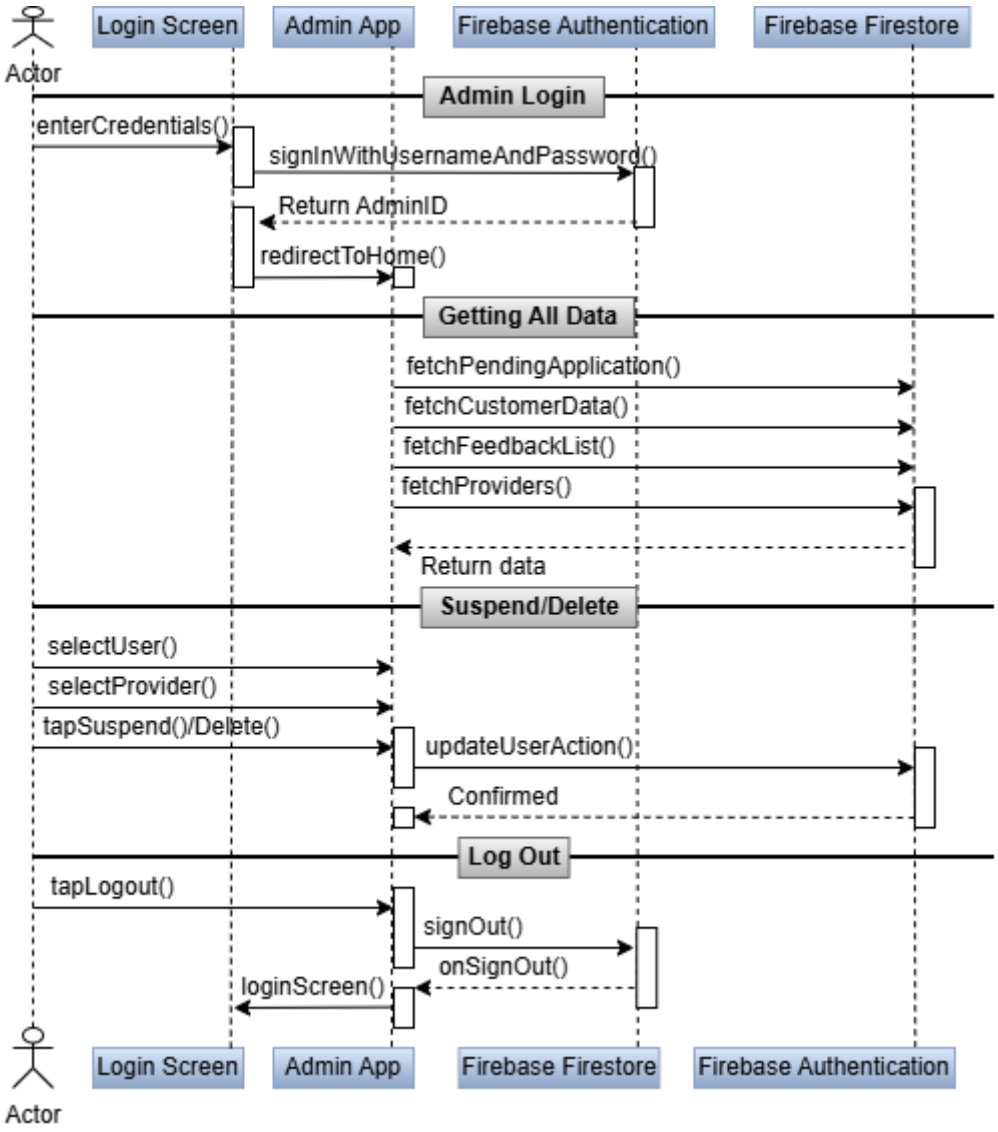


Figure 9 Sequence Diagram 7



3.2.3 State Diagram

The state diagram is used to model the dynamic behavior of a single entity, detailing the sequence of conditions or states an object can be. It maps how an object moves from one state to another state in response to specific events. For Example, it tracks a “Service Request” as it progresses the “Requested” state to “Accepted” and then finally to “completed”. This diagram is vital for precisely defining the lifecycle and rules that govern critical application entities.

Figure 11 State Diagram 1

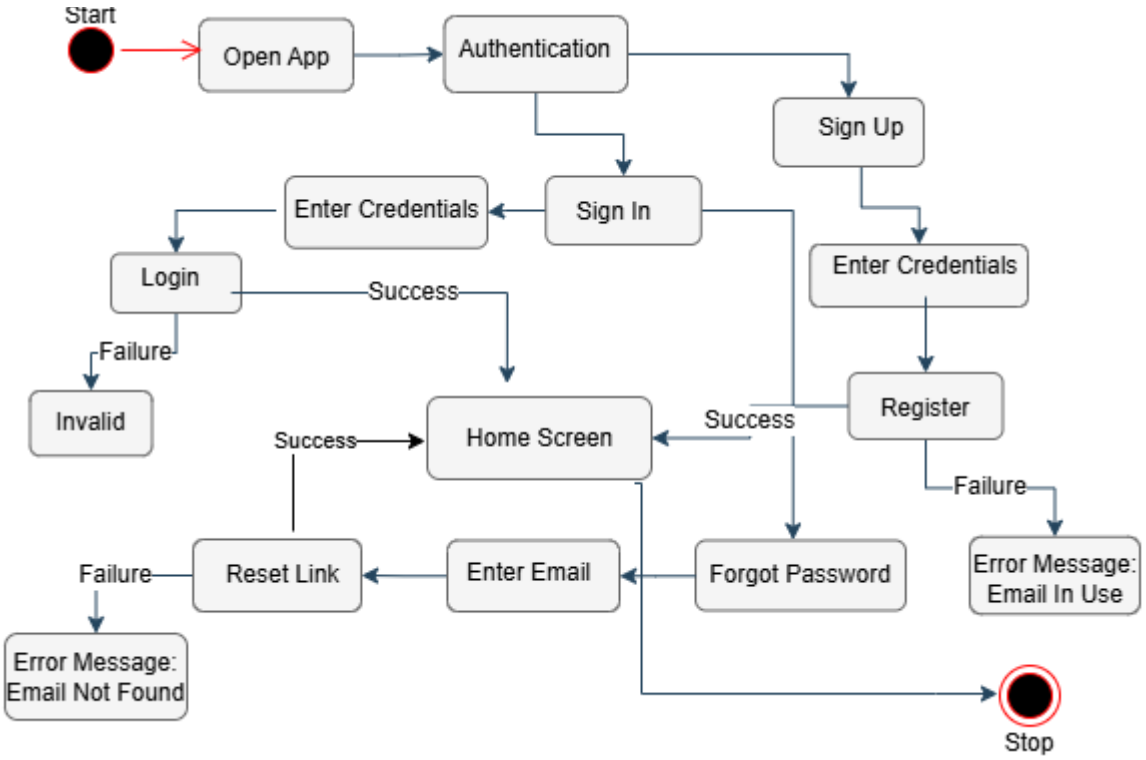


Figure 10 State Diagram 2

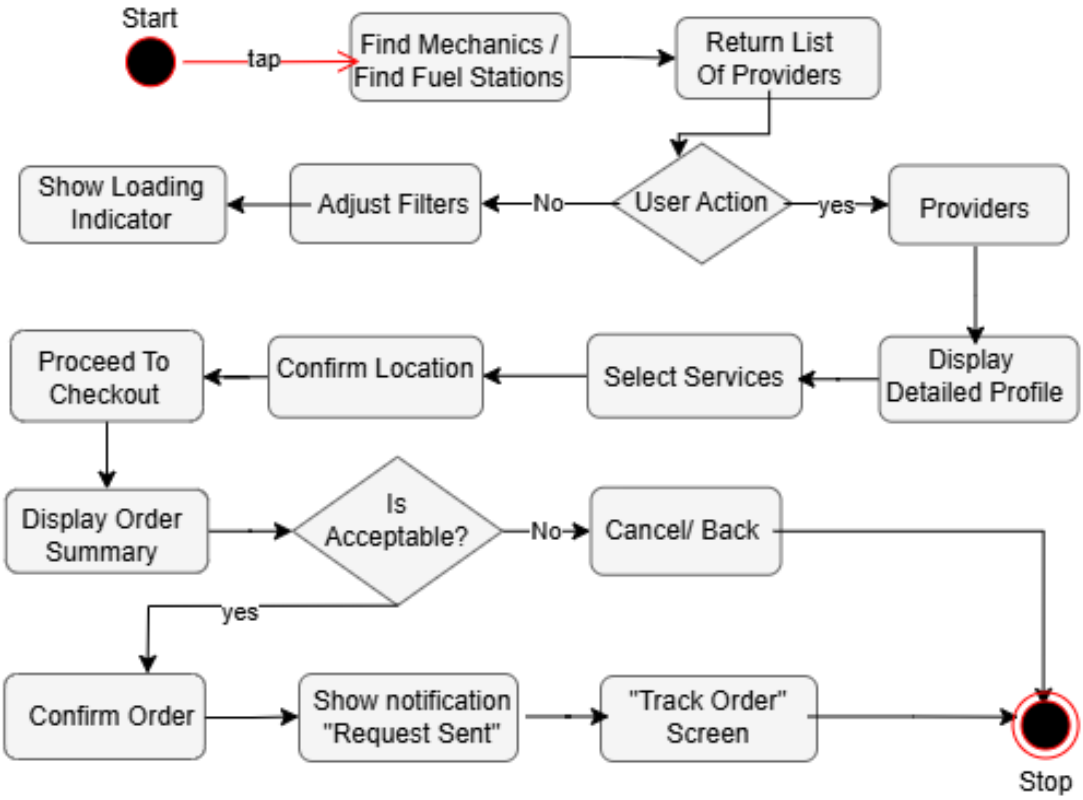


Figure 12 State Diagram 3

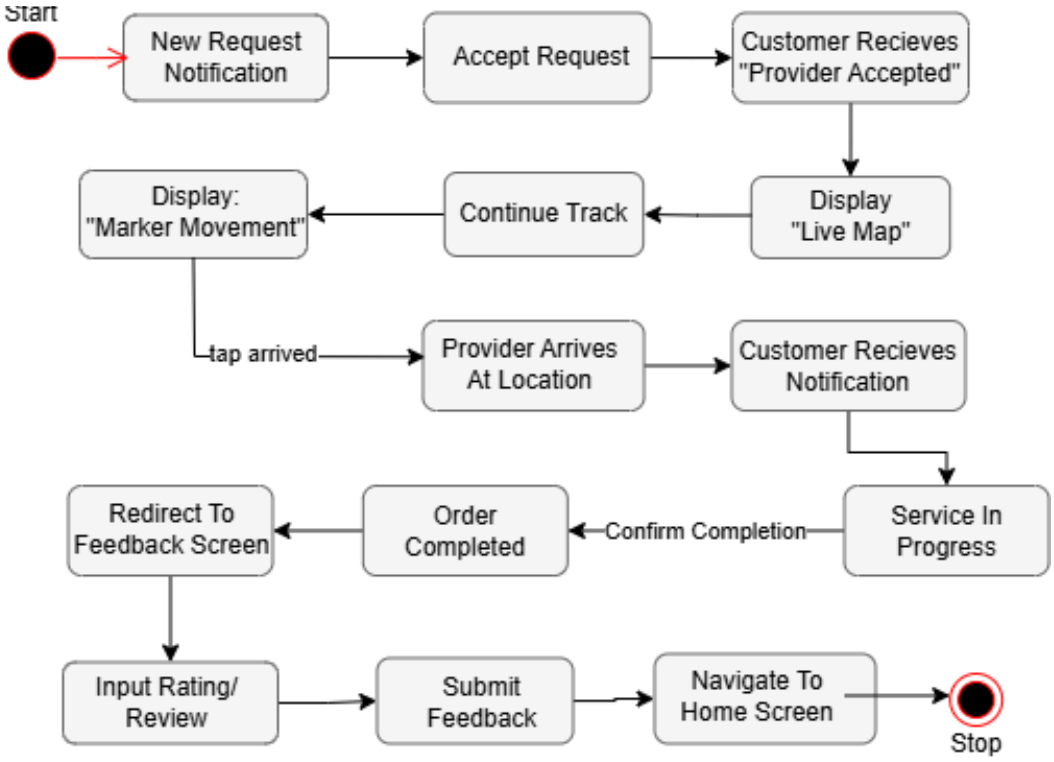


Figure 13 State Diagram 4

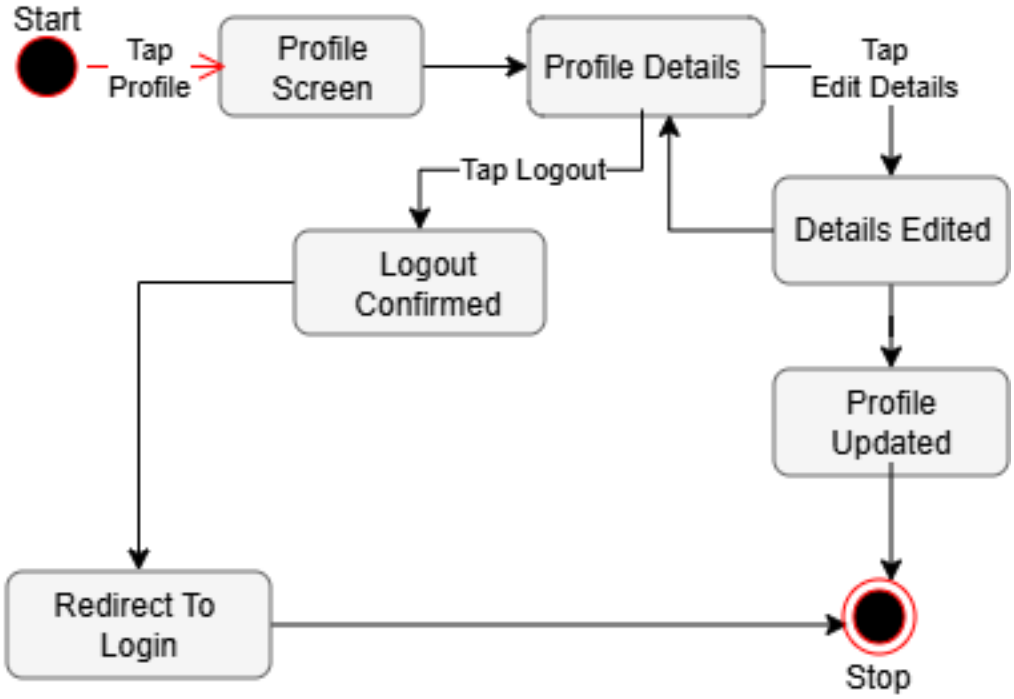


Figure 15 State Diagram 5

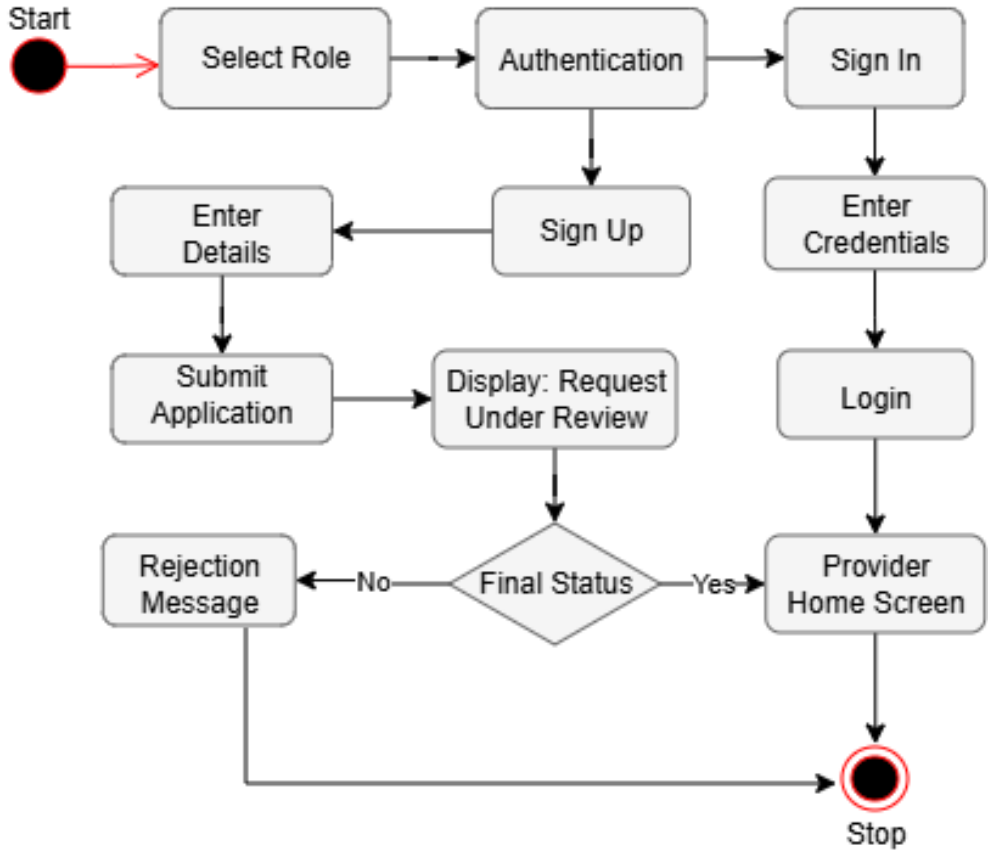


Figure 14 State Diagram 6

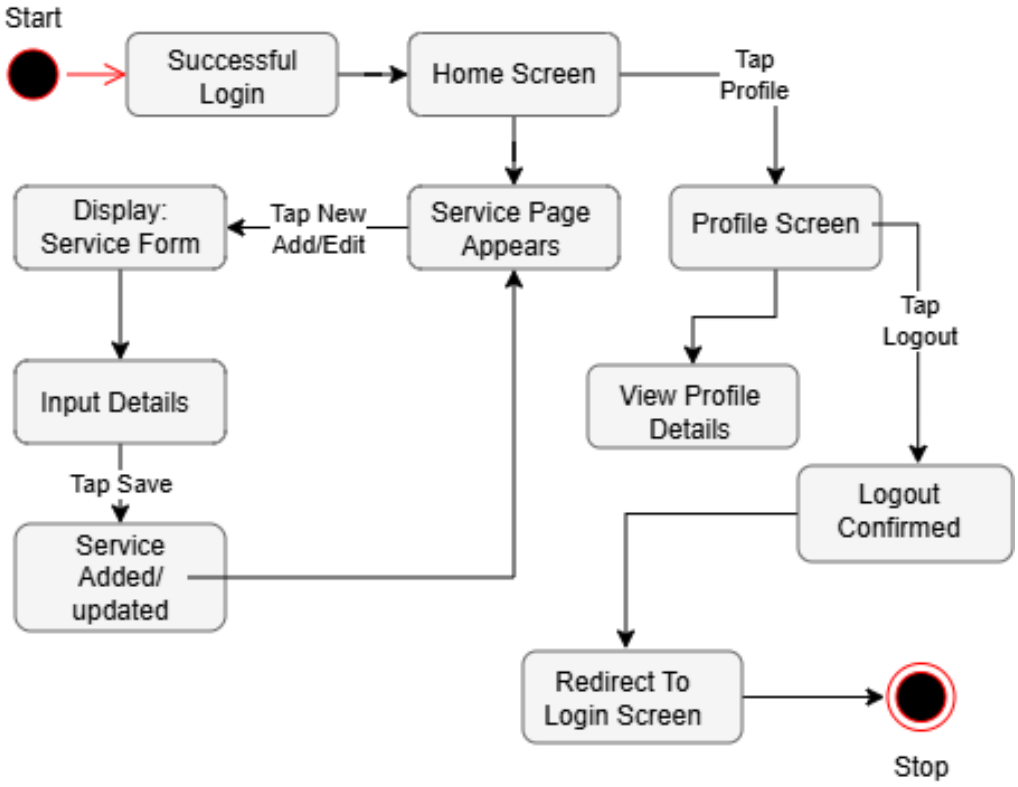
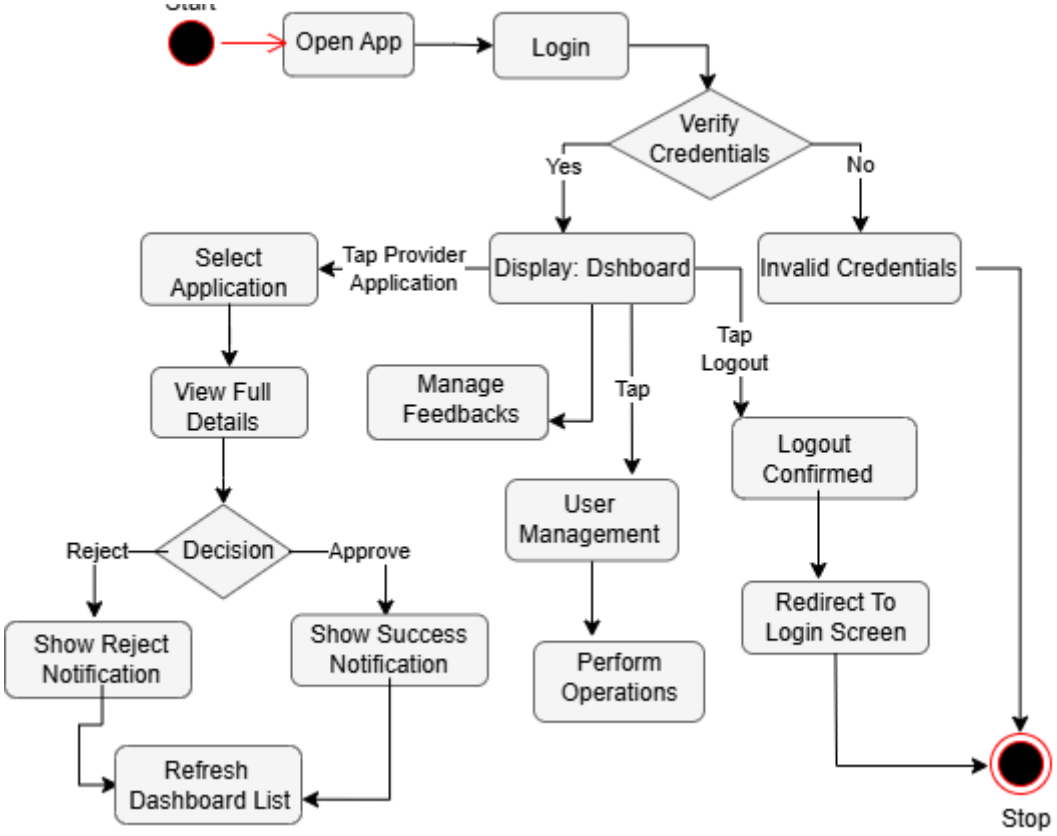


Figure 16 State Diagram 7



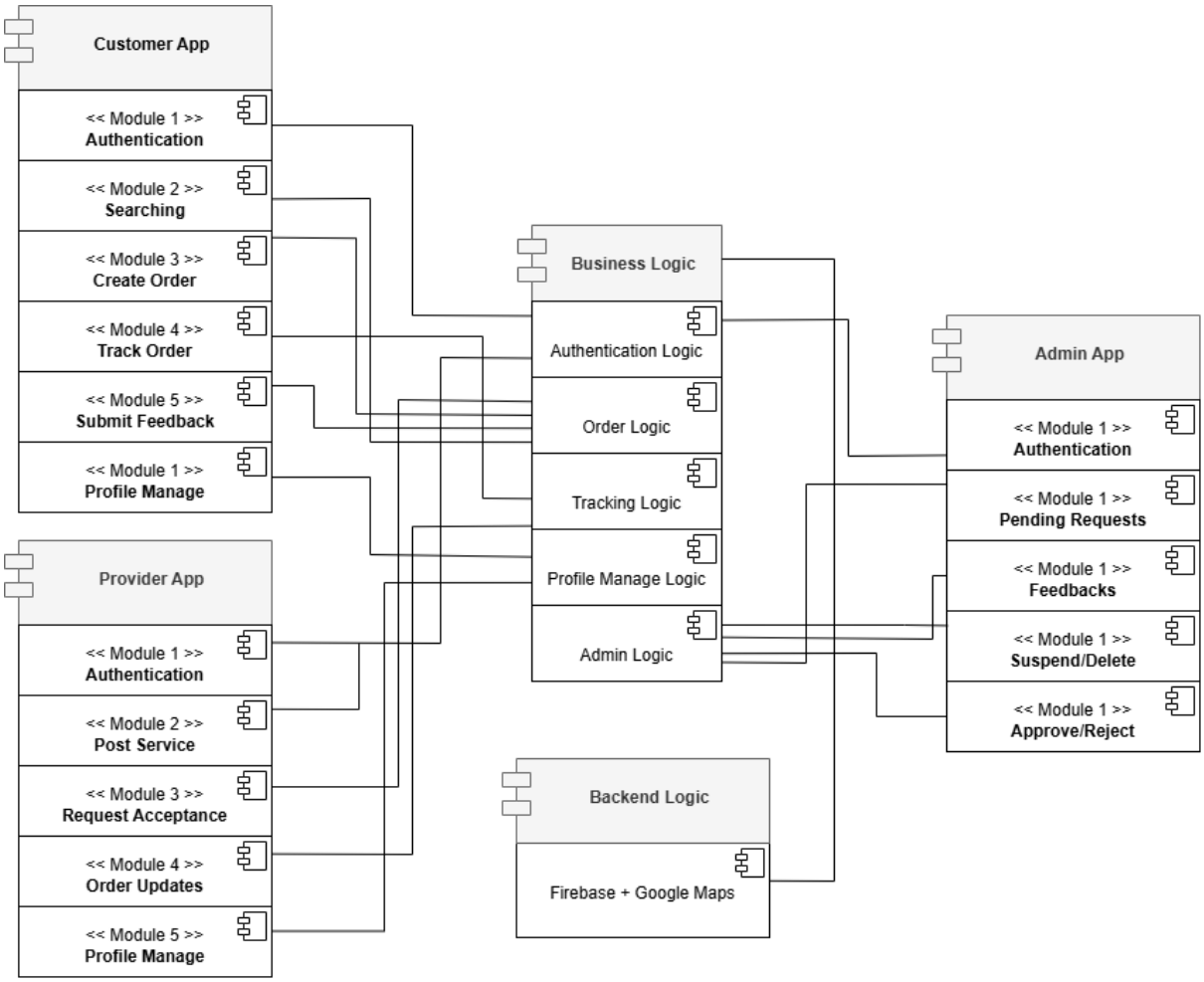
3.3 Architectural Design

Architectural design is concerned with understanding how a software system should be organized and designing the overall structure of that system. Architectural design is the first stage in the software design process. It is the critical link between design and requirement engineering, as it identifies the main structural components in a system and the relationship between them. The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components. The UML class diagram is already given in UML diagrams, here we will discuss component and architectural pattern diagram.

3.3.1 Component Diagram

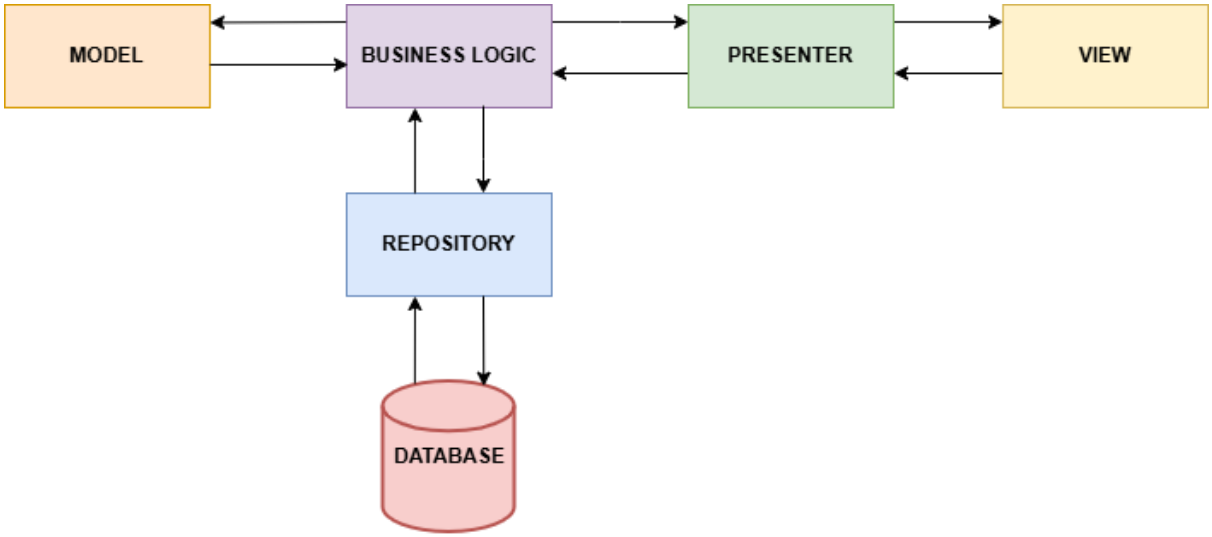
Component diagram illustrates the overall structure of the system by separating responsibilities across customer module, provider module and admin module, all connected to shared external device. Customer and provider application interact with firebase authentication for user login, firestore for data storage and retrieval and firebase cloud functions are used only where backend is required such as sending requests and processing updates. FCM support real-time notification between users. On admin side, dashboard connects to the firestore and authentication to manage provider approvals, feedbacks and account actions. This architecture ensure clear separation of responsibilities while enabling consistent interaction among all system components.

Figure 17 Component Diagram



3.3.2 Architectural Pattern Diagram

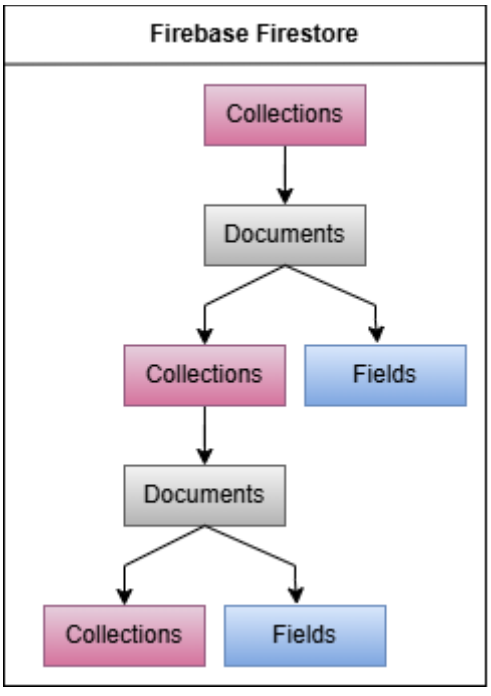
Figure 18 Architectural Pattern Diagram



3.4 Data Design

For database, we are using firebase which is No-SQL based object oriented backend service, which uses hierarchy of collections, documents and fields. The firebase is organized as a tree of collection, documents, under that again collections, and fields, which can further contain documents, and fields and collections. The firestore hierarchy is as given below:

Figure 19 Firestore Hierarchy



3.4.1 Data Dictionary

The data dictionary is comprised of classes, major classes are given below.

Table 45 Data Dictionary

Terminology	Description
Customer	Class: Represents the end-users, can register, login, search mechanics, fuel stations, book fuel delivery, send breakdown request, track request status and post feedback.
Admin	Class: Represents admin, view pending requests, approve/reject requests, view all registered users, view feedbacks, disable or delete account.
Mechanic	Class: Can register, login, post details, receive breakdown requests, update requests status, view feedbacks.
Fuel Station	Class: Can register, login create/ update fuel station details, receive fuel delivery bookings and update delivery status.
Breakdown Request	Class: used to store the details of user service request including description, location and tracking status.
Fuel Request	Class: used to store the details of fuel delivery bookings including quantity, location and delivery status.
Feedback	Class: used to store customer feedbacks.
Location	Class: used to store GPS coordinates for end users.

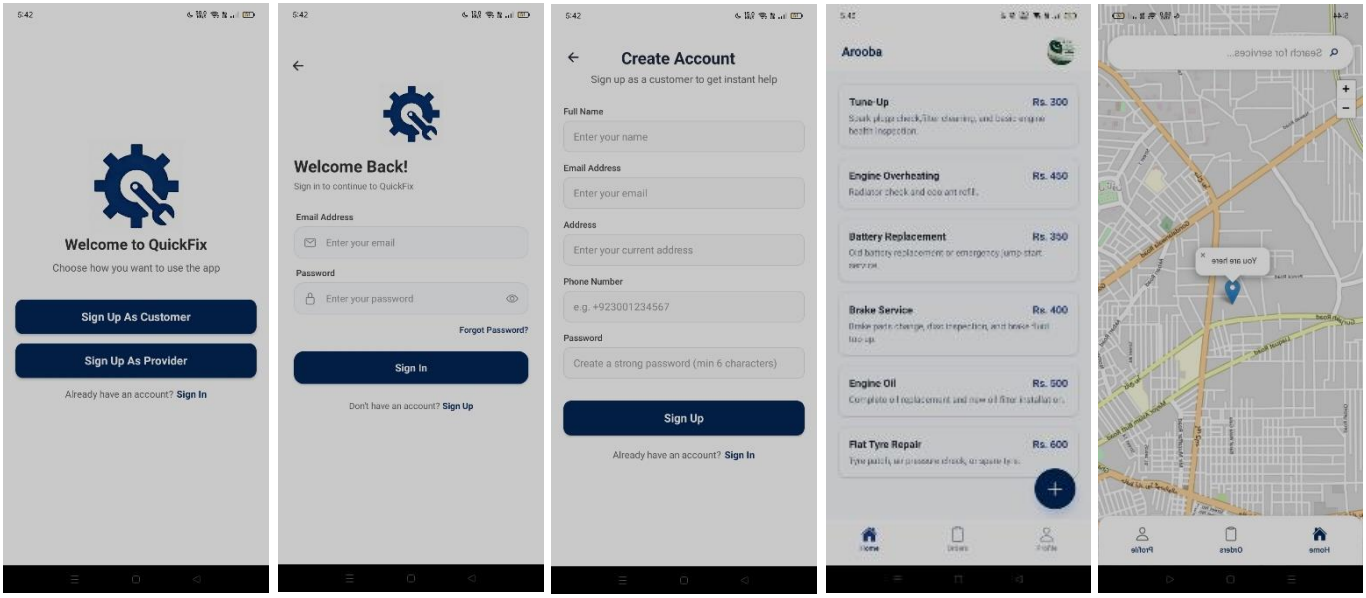
Firebase Authentication	Class: used to interact with firebase authentication.
Firebase Firestore	Class: used to interact with firebase firestore for storing and retrieving data.
FCM token	Class: unique identifier generated by firebase, used to send push notifications.
Document Snapshot	Class: used to interact with documents retrieved from firestore.

3.5 User Interface Design

The user interface is made using core methodologies and technologies, the whole flow of the application is designed with collective efforts of team with focus.

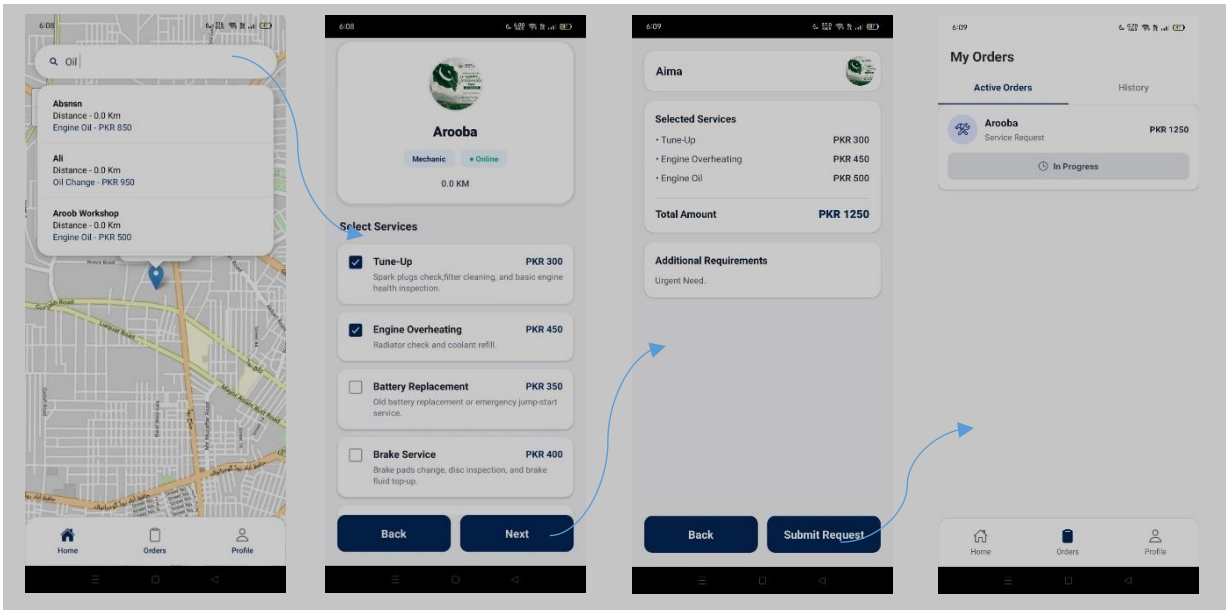
3.5.1 Screen Images

Figure 20 Screen Images



3.5.2 Screen Objects and Actions

Figure 21 Screen Objects and Actions



3.6 Design Decisions

For architecture we used MVP architecture pattern because it's easy to implement, testable and modular. For database we use firestore, which is No-SQL based Object Oriented database, highly compatible with mobile applications and also easy to use. From firebase we used only Firebase Firestore, Firebase Authentication and Firebase cloud messaging and firebase cloud functions only where needed. Firebase authentication provides multiple options for authentication, such as email and password authentication, password less authentication etc.

We will used React Native Map Libraries for displaying interactive maps on UI, enable real-time tracking feature between customer and provider. We utilize OpenStreetMaps for location based matching also.

We used Vs code for development and React Native was choose to bridge the gap between cross platform flexibility. Chooses a single set of code for both the Apple and Android Apps. This focus clears that we can finish building the apps much faster and we won't have to spend much time in fixing problems and adding new features on separate system. This simplicity makes the whole project easier to manage.

3.7 Summary

QuickFix was designed with a focus on modularity, scalability and maintainability. Features were divided using a component based approach in React Native, while an MVP structure kept the user interface and business logic separate for better code organization.

Chapter 4

Implementation

4. Implementation

This chapter describes the implementation part of project, before starting create a project in Firebase, makes sure its packages name matches the project package name created in React Native with JavaScript language. Enable firebase authentication with email and password, include firebase dependencies and synchronize the project.

This chapter will provide major logic's algorithms to help build a complete app.

4.1 Algorithms

QuickFix uses several algorithms across its core modules. The primary logic involves user authentication, service request and tracking management, code execution handling. Below are the pseudocode representations of the major components.

4.1.1 Algorithm 1 – Customer Registration

Section	Details
Algorithm	Customer Registration(Auth + Firestore)
Input	Name, Email, Address, Password
Output	Registered Data / Authentication Error

Pseudo Code

function RegisterCustomer(name, email, phone, address, password)

1. Check if name, email, address, or password are empty.
2. If any required field is empty, return "Validation Error: Missing Field".
3. userCredential ← Firebase.Auth.createUserWithEmailAndPassword(email, password)
4. If userCredential is null, return "Authentication Error".
5. uid ← userCredential.user.uid
6. token ← userCredential.user.getIdToken()
7. Firestore.Collection("Customers").Doc(uid).Set({
   "uid": uid,
   "name": name,
   "email": email,

"address": address,

"createdAt": Server.Timestamp()

})

8. Return { authTokens, userdata }

4.1.2 Algorithm 2 – Sign In

Section	Details
Algorithm	User Login (Firebase Authentication)
Input	Email, Password, Role
Output	Auth Token/ Authentication Error

Pseudo Code

function UserLogin(email, password, Role)

1. Check if email or password are empty.
2. If empty, Return "Email and Password are required".
3. UserCredentials ← Firebase.Auth.signInWithEmailAndPassword(email, password)
4. If userCredential is null, Return "Invalid Credentials".
5. uid ← userCredential.user.uid
6. If role = "customer" then

   userDoc ← Firestore.Collection("Customers").Doc(uid).Get()

   Else if role = "mechanic" then

   userDoc ← Firestore.Collection("Mechanics").Doc(uid).Get()

   Else if role = "fuel_station" then

   userDoc ← Firestore.Collection("FuelStations").Doc(uid).Get()

   Else

   Firebase.Auth.signOut()

   Return "Invalid Role"

7. If userDoc does not exist then

- Firebase.Auth.signout()
- Return “Error: Account not registered as” + Role

8. Token ← userCredential.user.getIdToken()
9. Return { authTokens, userdata }

4.1.3 Algorithm 3 – Search Nearby Providers

Section	Details
Algorithm	Provider Search
Input	Customer Location
Output	List of Nearby Available Providers

Pseudo Code

function SearchNearbyProviders(customerLat, customerLong, serviceCategory, radius)

1. Define minLat, maxLat, minLong, MaxLong based on the radius and customer coordinates.
2. Initialize nearbyProviderList as an empty list.
3. If serviceCategory = "mechanic" then
   providerCollection ← "Mechanics"

   Else if serviceCategory = "fuelStation" then

   providerCollection ← "FuelStations"

   Else Return empty list.
4. Fetch providers from Firestore.Collection(providerCollection)
   where: availabilityStatus == "online" isAvailable == true
5. For each provider:

   Calculate distance from customer.
   If distance <= radius: Add provider to nearbyProvidersList.
6. Sort nearbyProvidersList by exactDistance in ascending order.
7. Return nearbyProvidersList.

4.1.4 Algorithm 4 – Service Selection

Section	Details
Algorithm	Service Selection
Input	ProviderID, Available Services, Description
Output	Selected Services and Details

Pseudo Code

function SelectService(providerID, selectedServices, description)

1. Fetch available services of provider from Firestore.Collection("ProviderServices").
2. Display available services to customer.
3. Customer selects one or more services.
4. If no service is selected:

   Return "Please select a service".
5. Calculate total price of selected services.
6. Store selected services, description, and provider details.

4.1.5 Algorithm 5 – Checkout

Section	Details
Algorithm	Service Checkout
Input	Selected Services, Description, Provider Details
Output	Confirmed Service Request / Failure

Pseudo Code

function ConfirmCheckout(selectedServices, description, provider)

1. Calculate totalAmount from selected service prices.
2. Display Order Summary:
   - Selected Services
   - Total Amount
   - Additional Requirements

3. Wait for customer to click "Submit Request".
4. If "Submit Request" is clicked:

   Create a new request in Firestore

   with customer, provider, selected services,

   description, totalAmount and status = "pending".

5. If request is created successfully:

   Return "Request Submitted Successfully".

   Else: Return "Failed to Submit Service Request".

4.1.6 Algorithm 6 – Request Tracking

Section	Details
Algorithm	Real-time Request Tracking
Input	RequestID, ProviderID, ProviderType
Output	Request Status and Provider Location Updates

Pseudo Code

function TrackRequest(requestID, providerID, providerType)

1. Start StatusListener on

   Firestore.Collection("ServiceRequests").Doc(requestID)
2. On status update:

   currentStatus ← request.status

   Update customer UI with currentStatus.
3. Select provider collection:

   If providerType = "mechanic":

   collection ← "Mechanics"

   Else: collection ← "FuelStations"
4. Start LocationListener on

   Firestore.Collection(collection).Doc(providerID).
5. On location update:

   Get provider latitude and longitude.

Update provider marker on map.

Calculate distance from customer.

6. If request status = "completed":

   Stop active listeners. Navigate to Feedback Screen.

4.1.7 Algorithm 7 – Provider Registration

Section	Details
Algorithm	Provider Registration
Input	Auth Details, Profile Info, Business Info, Documents, Provider Type
Output	Provider Account Status(Admin Approval)

Pseudo Code

function RegisterProvider(authData, profileData, businessData, documentFiles , providerType)

1. Create Authentication Account:

   userCredential ← Firebase.Auth.createUserWithEmailAndPassword(authData.email,

   authData.password)

   providerUID ← userCredential.user.uid

2. Select Provider Collection:

   If providerType = "mechanic":

   collection ← "Mechanics"

   Else if providerType = "fuelStation":

   collection ← "FuelStations"

3. Create Provider Profile:

   providerData ← {

   "uid": providerUID,

   "name": profileData.name,

   "email": authData.email,

   "address": profileData.address,

"businessDetails": businessData,

"status": "pending"

}

4. Upload required documents to Firebase Storage and get their download URLs.
5. Add document URLs to providerData.
6. Save providerData in: Firestore.Collection(collection).Doc(providerUID)
7. Return "Registration Successful: Awaiting Admin Approval"

4.1.8 Algorithm 8 – Service Creation

Section	Details
Algorithm	Provider Service Creation
Input	ProviderID, ProviderType, Service Data
Output	Created Service / Error

Pseudo Code

function CreateProviderService(providerID, providerType, serviceData)

1. Select Provider Collection:

   If providerType = "mechanic":

   collection ← "Mechanics"

   Else: collection ← "FuelStations"

2. Get provider status from:

   Firestore.Collection(collection).Doc(providerID)

3. If provider.status != "approved" then

   Return "Error: Account is not approved."

4. Set newService ← {

   "providerId": providerID,

   "providerRole": providerType,

   "title": serviceData.title,

   "price": serviceData.price,

   "description": serviceData.description

}

5. Add newService to:

   Firestore.Collection("ProviderServices").Add(newService)

6. Return “Service Added Successfully”.

4.1.9 Algorithm 9 – Service Updates

Section	Details
Algorithm	Provider Service Updates
Input	RequestID, Action
Output	Updated Request Status

Pseudo Code

function ManageServiceUpdates(requestID, action)

1. If action == "Accept" then
   Update Firestore.Collection("ServiceRequests")
   .Doc(requestID) with:
   "status": "accepted"

   Return "Request Accepted"

2. else if action == "Reject" then
   Update Firestore.Collection("ServiceRequests")
   .Doc(requestID) with:
   "status": "rejected"

   Return "Request Rejected"

3. Else if action == "Complete" then
   Update Firestore.Collection("ServiceRequests")
   .Doc(requestID) with:
   "status": "completed"

   Return "Service Completed"

4. Else Return "Invalid Action"

4.1.10 Algorithm 10 – Manage Profile

Section	Details
Algorithm	Profile Management
Input	UserID, UserType, UpdatedData, ImageFile
Output	Updated Profile

Pseudo Code

function ManageProfile(userID, userType, updatedData, imageFile)

1. Select Collection:
   If userType == "customer":
   collection ← "Customers"
   Else if userType == "mechanic":
   collection ← "Mechanics"
   Else:
   collection ← "FuelStations"
2. If imageFile is not null then:
   Upload imageFile to Firebase Storage.
   Get profileImageURL.
3. Set updateObject ← {
   "name": updatedData.name,
   "phone": updatedData.phone,
   "updatedAt": Server.Timestamp()
   }
4. If profileImageURL exists then:
   Add "profilePic": profileImageURL
   to updateObject.
5. Update:
   Firestore.Collection(collection)
   .Doc(userID) with updateObject.
6. Return "Profile Updated Successfully".

4.1.11 Algorithm 11 – Notification & Feedback

Section	Details
Algorithm	Notification and Feedback System
Input	UserID, ProviderID, RequestID, Message, Rating, FeedbackText
Output	Notification/ Saved Feedback

Pseudo Code

function HandleNotificationAndFeedback(action, params)

1. if action == “NOTIFY” then

```
Create notificationData ← {
    "userId": params.userID,
    "message": params.message,
    "createdAt": Server.Timestamp()
}
Save notificationData in:
Firestore.Collection("notifications").Add(notificationData)
Send push notification using FCM.
```

2. else if action == “SUBMIT_FEEDBACK” then

```
Create feedbackData ← {
    "customerId": params.customerID,
     "customerName": params.customerName,
     "providerId": params.providerID,
     "providerName": params.providerName,
     "requestId": params.requestID,
     "rating": params.rating,
     "feedbackText": params.feedbackText,
     "createdAt": Server.Timestamp()
}
Save feedbackData in:
Firestore.Collection("feedback").Add(feedbackData)
```

Return "Feedback Submitted Successfully"

4.2 External APIs/SDKs

QuickFix uses OpenStreetMap with Leaflet for map and location-based functionalities. The only widely used external component is Firebase, which is added through dependencies classes and upon addition it can be accessed just like other React Native library classes.

4.3 Code Repository

Github Repository Link: https://github.com/aroobaansari13/Quick-Fix-Project

The complete source code of the QuickFix application is maintained in a public Git repository. The repository is updated as the application is tested, bugs are fixed, and existing features are improved.

4.4 Summary

This chapter outlined almost all major component's algorithms, although not all are included here in this chapter, as there's a lot of code. But one can get the understanding and working and flow of the things and understands the things from repository.

Chapter 5

Testing and Evaluation

5. Testing and Evaluation

This chapter focuses on testing and evaluation of our project. The validation and verification of different components will be considered. A structured testing approach is followed covering Unit Testing, Functional Testing, Integration Testing and Performance Testing.

5.1 Unit Testing

Unit testing in QuickFix validates individual units of code in isolation. These tests target specific methods within the Authentication, Service Requesting, Tracking, Creating Services, Providing Updates and Rating Modules to confirm that each function perform exactly as designed.

Unit Test Case 1

Table 46 Unit Test Case UT1

Test case ID	UT1
Requirement ID	R1
Title	Customer Registration, Valid Input
Description	Tests the customer registration function with valid name, email, address, and password.
Objective	Verify that a new customer account is successfully created through Firebase Authentication and customer data is stored in Firestore.
Precondition	Firebase Authentication and Firestore are available. No account already exists with the given email address.
Test Steps	1. Enter valid name, email, address, and password. 2. Execute the customer registration function. 3. Check Firebase Authentication for the created account. 4. Check the Customers collection for the customer document. 5. Verify that the generated UID and stored customer data are valid.

Input	name= “Sara”, email= sara123@gmail.com, password = “sara@&123”, address = “abc”
Expected Results	A new customer account is created successfully. A corresponding document is stored in the Customers collection with unique id.
Actual Result	Account created successfully and customer data stored correctly.
Remarks	Pass

Unit Test Case 2

Table 47 Unit Test Case UT2

Test case ID	UT2
Requirement ID	R2
Title	User Login, Correct Credentials
Description	Tests the login functionality using Firebase signInWithEmailAndPassword() and retrieves the corresponding role-based profile from Firestore.
Objective	Verify that an existing user can successfully authenticate with valid credentials and access the appropriate dashboard.
Precondition	A valid account exists in Firebase Authentication and in the corresponding Firestore collection. The user is currently logged out.
Test Steps	1. Enter valid email and password. 2. Execute the login function. 3. Verify Firebase Authentication returns a valid authenticated user. 4. Retrieve the user profile from the relevant collection (Customers, Mechanics, FuelStations).

	5. Verify that the application redirects the user to the correct dashboard.
Input	email= "sara123@gmail.com", password= "sara@&123"
Expected Results	The user is successfully authenticated. The system retrieves the user's specific role and redirects to the appropriate dashboard.
Actual Result	User authenticated successfully and correct screen was displayed.
Remarks	Pass

Unit Test Case 3

Table 48 Unit Test Case UT3

Test case ID	UT3
Requirement ID	R2
Title	User Login, Incorrect Password
Description	Tests the error-handling logic of the Login function when an incorrect password is entered for a registered email address.
Objective	Verify that the system prevents access when an incorrect password is provided for a valid email.
Precondition	A valid account for sara123@gmail.com exists in the system. The user is on the Login screen.
Test Steps	1. Enter the valid email and an incorrect password. 2. Call the Login() function. 3. Verify that Firebase Authentication rejects the login attempt. 4. Check that an appropriate error message is displayed.
Input	email="sara123@gmail.com", password="wrong123@$"

Expected Results	Authentication fails, an appropriate error message is displayed, the user remains on the Login screen, and no authenticated session is created.
Actual Result	Login was rejected and an error message was displayed.
Remarks	Pass

Unit Test Case 4

Table 49 Unit Test Case UT4

Test case ID	UT4
Requirement ID	R4
Title	Fetch Nearby Providers Logic
Description	Tests the nearby provider search logic by calculating the distance between the customer and available providers using their stored coordinates.
Objective	Verify that the system correctly identifies nearby available providers and sorts them according to their distance from the customer.
Precondition	Providers exist in Firestore with stored latitude and longitude. Providers are online and the customer's location is available.
Test Steps	1. Initialize search using customer coordinates. 2. Fetch available mechanics from the Mechanics collection. 3. Calculate the distance between the customer and each provider. 4. Filter providers outside the specified 7 km radius. 5. Sort the remaining providers by distance in ascending order.
Input	Customer Location = {lat: 31.52, long: 74.35}, Radius = 7km, Category = “mechanic”.
Expected Results	Only available providers are return within 7 Km radius.

Actual Result	Nearby available mechanics were retrieved and displayed according to their distance.
Remarks	Pass

Unit Test Case 5

Table 50 Unit Test Case UT5

Test case ID	UT5
Requirement ID	R5
Title	Service Request Creation
Description	Tests the createServiceRequest() method to verify that the customer's selected services and request information are correctly stored in Firestore.
Objective	Verify that a service request is successfully created with the selected services, total amount, customer and provider information, and "Pending" status.
Precondition	Customer is logged in, required services are selected, and the selected provider is online.
Test Steps	1. Select one or more services from an available provider. 2. Submit the service request. 3. Verify that a new request document is created in Firestore. 4. Verify the selected services and calculated total amount. 5. Verify that request status is "Pending".
Input	Selected Services = "Oil Change" Price = "400" Description = "Engine Oil Change Required"
Expected Results	A service request with a unique ID is created with the correct customer, provider, selected service, total amount, description.
Actual Result	Service request was created successfully with the correct details and pending status.

Remarks	Pass

Unit Test Case 6

Table 51 Unit Test Case UT6

Test case ID	UT6
Requirement ID	R5
Title	Checkout Total Calculation
Description	Tests the checkout calculation logic to ensure that the total amount of all selected services is calculated correctly.
Objective	Verify that the checkout screen correctly calculates and displays total price of selected services before the customer submits request.
Precondition	Customer has selected one or more services and proceeded to the Checkout screen.
Test Steps	1. Select multiple services. 2. Proceed to the Checkout screen. 3. Calculate the sum of selected service prices. 4. Verify the displayed total amount.
Input	Service1 = 1000, Service2 = 550
Expected Results	Selected services are displayed with their individual prices and the total amount is displayed as PKR 1550.
Actual Result	The checkout screen displayed the selected services and calculated the total amount as PKR 1550.
Remarks	Pass

Unit Test Case 7

Table 52 Unit Test Case UT7

Test case ID	UT7

Requirement ID	R9
Title	Service Creation Testing
Description	Tests the service creation functionality to ensure that the service title, description, price, and provider information are correctly stored in the ProviderServices collection.
Objective	Verify that a provider can successfully create a new service and that the service is associated with the correct provider.
Precondition	Provider is logged in and the provider account is approved.
Test Steps	1. Click the Add Service button. 2. Enter the service details and save. 3. Verify new service document in ProviderServices collection. 4. Verify that the stored providerId matches the logged-in provider.
Input	serviceName = “Oil Change”, Price = 150.00 PKR, Description = “Engine Oil Replacement”.
Expected Results	The service is saved successfully with a link to the providerID. The service now appears in the specific category search results.
Actual Result	Service was created and displayed in provider's service list.
Remarks	Pass

Unit Test Case 8

Table 53 Unit Test Case UT8

Test case ID	UT8
Requirement ID	R13
Title	Feedback and Rating Submission

Description	Tests the feedback submission functionality to ensure that the customer's rating and feedback are correctly stored in Firestore for the selected provider.
Objective	Verify that a customer can successfully submit a rating and feedback after completion of a service request.
Precondition	Service request has been completed and the customer is on the Feedback screen.
Test Steps	1. Enter a 5-star rating and text comment. 2. Click “Submit Review”. 3. Verify that a new feedback record is created in the Feedbacks collection. 4. Verify that the rating, feedback text, provider ID, customer ID, and request ID are stored correctly.
Input	Rating = 5, comment = “Good Work!”
Expected Results	Feedback is successfully stored in Firestore with the correct rating, feedback text, customer, provider, and request information.
Actual Result	Feedback and rating were submitted and stored successfully.
Remarks	Pass

5.2 Functional Testing

Functional testing verifies that all system features works correctly according to the specified requirements. It focuses on testing complete functionalities such as authentication, provider search, booking request, live tracking, notifications and feedback from the user’s perspective. The goal is to ensure that each feature produces the expected output for given inputs without considering internal code.

Functional Test Case 1

Table 54 Functional Test Case FT1

Test case ID	FT1

Requirement ID	R7
Title	Provider Account Creation
Description	Tests the provider registration to verify that the information and required documents are successfully submitted and stored.
Objective	Verify that a provider can register and submit documents for admin approval.
Precondition	Provider is not registered and required details/documents are available.
Test Steps	1. Open Provider Registration. 2. Enter required details. 3. Upload business documents. 4. Submit registration. 5. Verify account data and pending approval status.
Input	name = “John Doe”, email = johndoe@gmail.com, address = “abc”, providerType = “Mechanics”
Expected Results	Account is created successfully and remains pending for admin approval.
Actual Result	Account created and submitted for approval.
Remarks	Pass

Functional Test Case 2

Table 55 Functional Test Case FT2

Test case ID	FT2
Requirement ID	R14
Title	Provider Verification
Description	Test admin approval of a registered provider.
Objective	Verify that Admin can review and approve a provider account.

Precondition	Admin is logged in and a provider with pending status exists.
Test Steps	1. Open Pending Applications. 2. Select the provider. 3. Review provider details/documents. 4. Click Approve. 5. Verify updated status in Firestore.
Input	providerID = “abx”, action = “approved”.
Expected Results	Provider status changes to Approved and the provider can access approved provider functionality.
Actual Result	Provider was successfully approved.
Remarks	Pass

Functional Test Case 3

Table 56 Functional Test Case FT3

Test case ID	FT3
Requirement ID	R13
Title	Live Location Tracking
Description	Tests provider location updates from the provider device to the customer map.
Objective	Verify that the provider’s live location is correctly displayed to the customer.
Precondition	Active service request exists and GPS is enabled on both devices.
Test Steps	1. Provider changes location. 2. Verify location update in Firestore. 3. Check updated provider location on the customer map.
Input	Latitude: 31.52, Longitude: 74.35

Expected Results	Provider's map marker updates according to the latest location and the displayed distance updates accordingly.
Actual Result	Provider location updated successfully on the customer map.
Remarks	Pass

Functional Test Case 4

Table 57 Functional Test Case FT4

Test case ID	FT4
Requirement ID	R16
Title	Customer Account Moderation
Description	Tests Admin functionality for managing customer accounts.
Objective	Verify that Admin can view and disable/delete a customer account.
Precondition	Admin is logged in and a customer account exists.
Test Steps	1. Open Customer Management. 2. Select the customer. 3. Click Disable/Delete Account. 4. Verify updated account status.
Input	customerID = "sara", action = "disabled".
Expected Results	Customer account status changes to Disabled/Deleted and access is restricted.
Actual Result	Customer account was successfully suspended/deleted.
Remarks	Pass

Functional Test Case 5

Table 58 Functional Test Case FT5

Test case ID	FT5
Requirement ID	R15

Title	Notification Trigger
Description	Tests FCM notification delivery when a customer submits a service request.
Objective	Verify that the provider receives a notification for a new service request.
Precondition	Provider has a valid FCM token and customer submits a service request.
Test Steps	1. Customer submits a service request. 2. Trigger the provider notification. 3. Verify notification is received on the provider device.
Input	requestID = "REQ_123" Provider FCM Token = Valid token
Expected Results	Provider receives a notification for the new service request.
Actual Result	Notification was successfully received by the provider.
Remarks	Pass

Functional Test Case 6

Table 59 Functional Test Case FT6

Test case ID	FT6
Requirement ID	R17
Title	Admin View and Moderate Feedbacks
Description	Tests Admin functionality to view and manage customer feedback.
Objective	Verify that Admin can view feedback and remove inappropriate feedback.
Precondition	Admin is logged in and customer feedback exists in the database.
Test Steps	1. Open the Feedbacks section. 2. Verify rating, feedback, customer and provider details.

	3. Select feedback and click Delete. 4. Verify it is removed from Firestore.
Input	Admin Action = "View All Reviews"
Expected Results	Feedback records are displayed correctly and selected feedback can be deleted successfully.
Actual Result	Feedback was displayed and deleted successfully.
Remarks	Pass

5.3 Integration Testing

Integration testing verifies that the individual components interacts correctly as a combined system. These tests focus on the interfaces between Front-end, Application server, Authentication service, Database Layer and Code Execution Engine.

Integration Test Case 1

Table 60 Integration Test Case IT1

Test case ID	IT1
Requirement ID	R1
Title	Frontend → Authentication → Database: User Login
Description	Tests integration between Login UI, Firebase Authentication, and Firestore.
Objective	Verify that valid login credentials authenticate the user and retrieve the correct profile data.
Precondition	Firebase is initialized and a valid user account exists.
Test Steps	1. Enter valid login credentials. 2. Submit the Login form. 3. Verify Firebase Authentication. 4. Retrieve the user UID and profile from the relevant Firestore collection.

	5. Verify navigation to the correct dashboard.
Input	email = sara123@gmail.com , password = “sara123&@”
Expected Results	User is authenticated, correct profile data is retrieved, and the appropriate dashboard opens.
Actual Result	Login and profile retrieval completed successfully.
Remarks	Pass

Integration Test case 2

Table 61 Integration Test Case IT2

Test case ID	IT2
Requirement ID	R3
Title	Customer → Firestore → Provider: Service Request
Description	Tests integration between customer service request, Firestore, and provider interface.
Objective	Verify that a customer request is stored and displayed to the selected provider.
Precondition	Customer and provider are logged in, and provider is online and available.
Test Steps	1. Customer selects a service and provider. 2. Submit the service request. 3. Verify request creation in Firestore. 4. Verify the request appears in Provider Orders.
Input	customerID = “CUST_001”, serviceType = “Tyre Burst”, providerID = “PROV_001”
Expected Results	Request is stored with pending status and appears on the selected provider's Orders screen.
Actual Result	Request was stored and displayed to the provider successfully.

Remarks	Pass

Integration Test Case 3

Table 62 Integration Test Case IT3

Test case ID	IT3
Requirement ID	R6
Title	Provider GPS → Firestore → Map Synchronization
Description	Tests integration between provider GPS, Firestore location updates, and customer map.
Objective	Verify that provider location updates are reflected on the customer's map.
Precondition	Location permission is granted and provider location tracking is active.
Test Steps	1. Change provider location. 2. Verify updated coordinates in Firestore. 3. Check provider marker on the customer map.
Input	Latitude = 31.67, Longitude= 74.56
Expected Results	Updated coordinates are stored in Firestore and the provider marker updates on the customer map without manual refresh.
Actual Result	Provider location and customer map were synchronized successfully.
Remarks	Pass

Integration Test Case 4

Table 63 Integration Test Case IT4

Test case ID	IT4
Requirement ID	R15

Title	Service Request → FCM → Provider Notification
Description	Tests integration between service request creation and FCM notification delivery.
Objective	Verify that a new service request triggers a notification to the selected provider.
Precondition	Provider has a valid FCM token and notifications are enabled.
Test Steps	1. Customer submits a service request. 2. Verify notification process is triggered. 3. Verify FCM delivers the notification. 4. Check notification on provider device.
Input	Event = "new request", TargetUID = "prov_1", FCM_token = Valid token
Expected Results	Notification is delivered to the correct provider with the new request information.
Actual Result	Provider received the notification successfully.
Remarks	Pass

Integration Test Case 5

Table 64 Integration Test Case IT5

Test case ID	IT5
Requirement ID	R7
Title	Service Selection → Checkout → Request Creation
Description	Tests integration between service selection, checkout total, and request creation.
Objective	Verify that selected services and their total amount are correctly displayed and stored in the service request.
Precondition	Customer has selected services and reached the Checkout screen.

Test Steps	1. Select services. 2. Open Checkout. 3. Verify total amount. 4. Submit request. 5. Verify request data in Firestore.
Input	Service 1 = 1600, Service 2 = 400
Expected Results	Checkout displays PKR 2000 and the same total with selected services is stored in the request.
Actual Result	Correct total and service details were stored successfully.
Remarks	Pass

5.4 Performance Testing

Performance testing evaluates the responsiveness, stability and resource utilization under normal and stress conditions. The testing focuses on two critical operations: Response time for common requests and code execution turnaround time.

Tools used: Manual Application Performance Testing

Table 65 Performance Testing Results

Tool/ Method	Metric Measured	Result	Observation
User Login	Response Time	420 ms	Fast and responsive
Nearby Provider Search	Response Time	510 ms	Providers displayed efficiently
Service Request Submission	Response Time	460 ms	Request created successfully
Provider Request Retrieval	Response Time	390 ms	Request displayed quickly
Feedback Submission	Response Time	350 ms	Feedback saved efficiently

Live Location Update	Update Delay	620 ms	Location updated smoothly
FCM Notification	Delivery Delay	1.1 sec	Notification delivered promptly
App Screen Loading	Average Load Time	2 sec	Screens loaded smoothly

Observations:

- QuickFix demonstrated good responsiveness and stability during major application operations.
- User authentication and provider search completed with acceptable response times.
- Service requests were created and displayed to providers without significant delay.
- Provider location updates were reflected smoothly on the customer's map.
- Firebase Cloud Messaging (FCM) delivered notifications with low delivery delay.
- Feedback submission and retrieval performed efficiently.
- Overall, the application remained stable and responsive during testing.

Table 66 Performance Testing PT1

Test case ID	PT1
Requirement ID	R1
Title	User Login Performance
Description	Tests the response time of user authentication in QuickFix.
Objective	Verify that users can log in within an acceptable response time.
Precondition	A valid user account exists and internet connection is available
Test Steps	1. Open Login screen. 2. Enter valid credentials. 3. Tap Sign In. 4. Observe the login response time and dashboard loading.
Input	Valid Email and Password
Expected Results	User authentication completes quickly and the correct dashboard opens without noticeable delay.
Actual Result	Login completed successfully with an observed response time of 420 ms.
Remarks	Pass

5.5 Summary

The testing phase validated QuickFix's functionality, integration, performance, and reliability. Unit, Functional, Integration, and Performance Testing confirmed that major features, including authentication, service requests, real-time location tracking, and notifications, operate successfully. Overall, QuickFix provides a stable and responsive environment for end users.

Chapter 6

System Conversion

6. System Conversion

System conversion is the process of introducing QuickFix to its intended users and transitioning from the existing manual service-finding process to the digital platform while minimizing operational issues.

6.1 Conversion Method

For the implementation of QuickFix service platform, the selected conversion method is:

6.1.1 Pilot Conversion

Pilot conversion introduces QuickFix to a small group of users and service providers before the full launch. This allows the team to monitor key features such as service requests and GPS tracking. Any technical issues can be identified and resolved during this stage. This ensures a smooth and reliable full-scale launch.

Justification:

Reduced Risk: Since only a small portion of the users will use the new system initially, the risk of system-wide failure is minimized.

Focused Testing: Allows in-depth testing of modules such as role-based authentication, matching logic and tracking.

User Feedback Loop: Initial users' feedback can be used to improve usability and performance before full deployment.

Pilot Scope:

Initial Group: A limited group of customers and verified service providers.

Modules Tested: Registration, Login, Provider Search, Service Requests, Notifications, and Real-time Location Tracking.

Duration: A limited testing period followed by full deployment after resolving identified issues.

6.2 Deployment

Deployment involves transferring the QuickFix application from the development environment to an operational environment for end users. It includes building the application, configuring Firebase services, and verifying that major application features work correctly.

6.2.1 Deployment Steps:

Table 67 Deployment Steps

Step No.	Task	Details
1	Production Build	Generate signed APK/AAB for Android and IPA for iOS for production deployment.
2	Firebase Configuration	Configure Firebase Authentication, Firestore, Storage, and FCM for production use.
3	Map Configuration	Configure OpenStreetMap and location services for provider search and tracking.
4	Multi-Device Testing	Test the application on different devices and screen sizes.
5	Live Environment Testing	Verify service requests, location tracking, notifications, and other major workflows.
6	Application Distribution	Distribute the application via Firebase App Distribution for final validation.

6.2.2 Data Conversion

Table 68 Data Conversion

Step	Task	Details
1	Data Preparation	Identify required customer, provider, service, request, feedback, and notification data.

2	Data Cleaning	Check existing data for missing, incorrect, or duplicate records.
3	Data Organization	Organize data according to the required Firestore collections and fields.
4	Data Verification	Verify that stored data is complete, accurate, and accessible by the application.

6.2.3 Training

A descriptive training video will be provided to demonstrate the main functions of QuickFix for customers and service providers.

Table 69 Training Instruction 1

Step	Instruction
1	Log in using your registered email and password.
2	Select a service category (e.g., Brake Inspection) from the home dashboard.
3	Select a nearby available provider and view their services..
4	Select required services, review the checkout details, and submit the request.
5	View the request status and track the provider’s location when applicable.

Table 70 Training Instruction 2

Step	Instruction
1	Log in to QuickFix and set your availability to Online.
2	Add or manage the services offered to customers.
3	View incoming customer requests in the Orders section.
4	Review the request and Accept or Reject it.
5	Complete the accepted service request after providing the service.

6.3 Post Deployment Testing

Post-deployment testing were conducted to validate that the QuickFix system is correctly installed, configured, and operating as intended. The objective was to ensure data integrity, feature functionality, and user experience after deployment.

6.3.1 Post Deployment Testing Steps

Table 71 Post Deployment Testing Steps

Step	Test Activity	Expected Outcome
1	Launch Application	Application launches successfully without errors.
2	Login Functionality	Registered users can successfully log in.
3	Service Request	Customers can successfully place a request and see it listed in “My Orders” Section.
4	Real-Time Tracking	Provider location updates correctly on the customer map.
5	Provider Response	Providers receive instant push notifications and can accept/reject jobs via the dashboard.
6	Role-Based Access	Customer, Provider, and Admin access their respective features correctly.
7	Notification Check	Notifications are successfully delivered to the intended user.
8	Performance Check	Application remains stable and responsive during normal use.

6.4 Challenges and Solutions

During Deployment and post-deployment phases of QuickFix, several technical and operational challenges were encountered. Below is a list of the main challenges along with their applied solutions:

Table 72 Challenges and Solutions

Challenge	Description	Solution Implemented
Device Compatibility	Some screens displayed differently on different Android devices.	Tested the application on different screen sizes and improved layouts using React Native Flexbox.
APK Installation	Installation issues occurred on some Android devices.	Generated and tested a signed APK for reliable installation.
Role-Based Access	Some users experienced incorrect access or Firestore permission issues.	Reviewed authentication flow and Firebase Security Rules for different user roles.
Real-Time Location Tracking	Provider location updates were delayed or inconsistent in some cases.	Improved Firestore listeners and location update logic for better synchronization.
Service Request Synchronization	Customer requests needed to appear correctly on the selected provider's Orders screen.	Improved Firestore request handling and real-time request listeners.
Notification Delivery	Notifications were not consistently delivered during some testing scenarios.	Verified FCM tokens and improved notification handling.
User Onboarding	New users required guidance for service requests and provider workflows.	Prepared training video explaining major application functions.

6.5 Summary

This chapter focuses on how our app is deployed and what training does user need, also how we converted data into information that is useful for our project. Also, it focuses on challenges faced during development and how these challenges were tackled.

Chapter 7

Conclusion

7. Introduction

This chapter will conclude what have been completed, partial, or still under development.

7.1 Evaluation

Table 73 Progress Table

Develop secure authentication for Customers, Service Providers, and Admin with role-based access.	Completed
Enable Service Providers to create, update, and delete service listings with description and price.	Completed
Allow Customers to search and view nearby available Service Providers and their offered services.	Completed
Enable Customers to select services, add requirements, review the total amount, and submit a service request.	Completed
Provide real-time Provider location tracking for Customers through an interactive map.	Completed
Send push notifications through FCM for important service request updates.	Completed
Allow Customers to submit ratings and feedback after service completion.	Completed
Enable Providers to receive and view incoming service requests.	Completed
Maintain service request/order information and status in real time through Firestore.	Completed
Allow Providers to accept, reject, and complete service requests.	Completed
Allow Admin to approve/reject Provider applications, manage accounts, and view/manage feedback.	Completed

7.2 Traceability Matrix

Table 74 Traceability Matrix Table

Req ID	**Requirement Description**	**Design Specification**	**Code File(s)**	**Test ID**

R1	Customer registers an account.	Registration, Firebase Authentication, Firestore	CustomerSignUp.js, AuthManager.js	UT1
R2	Users securely log in using email and password.	Authentication, Role-Based Login	SignIn.js, AuthManager.js	UT2, UT3
R3	User resets a forgotten password.	Forgot Password, Firebase Authentication	ForgotPasswordModal.js	-
R4	Customer searches for nearby available providers.	Provider Search, Location Services	ProviderSearchService.js, CustomerHome.js	UT4
R5	Customer selects services and creates a service request.	Service Selection, Firestore	ServiceRequestService.js	UT5, IT2
R6	Provider's location is tracked on the customer map.	Location Tracking, OpenStreetMap, Firestore	locationService.js, TrackingMap.js	FT3, IT3
R7	Customer reviews checkout details and confirms the request.	Checkout, Cost Calculation	CheckoutScreen.js,	UT6, IT5
R8	Customer views service request/order status.	Request Tracking, Firestore	TrackingMap.js. CustomerOrders.js	IT2
R9	Provider creates and manages offered services.	Service Management, Firestore	MechanicHome.js, FuelStationHome.js	UT7
R10	Provider registers with personal and business information.	Provider Registration, Firebase	Provider Sign-Up files	FT1

R11	Provider receives incoming service requests.	Provider Orders, Firestore Listener	ProviderOrders.js, RequestCard.js	IT2
R12	Provider accepts or rejects service requests.	Request Management, Firestore	ServiceRquestDetails.js, ServiceRequestService.js	IT2
R13	Customer submits rating and feedback after service completion.	Feedback, Firestore	FeedbackModal.js	UT8
R14	Admin reviews and approves/rejects Provider registrations.	Provider Verification, Firestore	AdminPendingApplications.js	FT2
R15	System sends notifications for important request events.	FCM, Notification Handling	functions/Index.js	FT5, IT4
R16	Admin views and manages Customer accounts.	Customer Management, Firestore	AdminCustomerDetails.js	FT4
R17	Admin views and manages Customer feedback.	Feedback Management, Firestore	AdminFeedbacks.js	FT6
R18	Customer and Provider can manage their profile information.	Profile Management, Firestore	CustomerProfile.js, ProviderProfile.js	-

7.3 Conclusion

Conclusion drawn from our work and results is that it was need of the hour and user accepted the app and used it and liked it.

7.4 Future Works

The following enhancements are proposed to improve the QuickFix system in future:

- Integrate online payment methods (e.g., cards, mobile wallets) for secure transactions.
- Add in-app chat and calling feature between customer and provider.
- Implement offline support with limited functionality using local storage.
- Add multi-language support to increase usability for wider audience.
- Improve security features such as two-factor authentication (2FA).
- Enhance scalability to support a larger number of concurrent users.
- Introduce service history and analytics dashboard for users and providers.
- Add emergency SOS feature for critical breakdown situations.

References

Web Resources

- (n.d.). Retrieved from RSA Pakistan: https://rsapakistan.com/
- Hussain, M. (n.d.). *SOW Programming*. Retrieved from Youtube: https://www.youtube.com/@SOWProgramming
- Khan, H. A. (n.d.). *Code With Harry*. Retrieved from Youtube: https://youtube.com/@codewithharry

Books

- Ian, S. (n.d.). Software Engineering. In *Pearon Education* (pp. 120-321).

Appendix A: Use Case Description

The detailed use case descriptions of the QuickFix system have already been presented in Chapter 2. To avoid duplication, they are not repeated in this appendix. For complete use case specifications, refer to Section **[2.5**Use Case Analysis**]** in Chapter 2. Below is just the Use Case Description Template is added just for the requirement of documentation.

Table 75 Use Case Description

Use Case ID	Unique numeric identifier for each use case e.g. UC-1, UC-2, UC-3.
Use Case Name	Enter the short name for the use case such as: Customer Registration, Login, Service Browse, Location Track, Profile Manage etc.
Actors	An actor is a person or other entity external to the software system being specified who interacts with the system and performs use cases to accomplish tasks.
Description	Provide a brief description of the reason and outcome of this use case.
Trigger	Identifies the event that initiates the use case.
Pre-Condition	List any activities that must take place, or any condition that must be true, before the use case can be started.
Post-Condition	Describe the state of the system at the conclusion of the use case execution.
Normal Flow	Provide a detailed description of user actions and system responses that will take place during execution of the use case under normal, expected flow.
Alternative Flow	Document legitimate branches from the main flow to handle special conditions.
Business Rules	Use cases and business rules are interviewed. Some business rule constrain which roles can perform all parts of a use case.
Assumption	List any assumption.

Appendix B: General Coding Standards & Guidelines

To ensure the readability, maintainability, and consistency of the QuickFix codebase, the following coding standards and guidelines were followed during development.

Variable Naming Conventions

- **PascalCase:** Used for React Native components, e.g., ProviderOrders.
- **camelCase:** Used for variables and functions, e.g., isAvailable, handleLogin().
- **SNAKE_CASE:** Used for constants, e.g., ADMIN_CREDENTIALS.

Boolean Naming

- **Boolean variables:** Boolean variables use meaningful prefixes such as is, has, and should, e.g., isAvailable and isVerified.

Variable Declaration

- **Const:** Use const for values that are not reassigned.
- **Let:** Use let only when reassignment is required.

Function Naming

- **Descriptive Naming**: Functions use descriptive names that clearly represent their purpose.
- **Action-Based Naming:** Action-based names such as handleLogin(), completeRequest(), and updateRequestStatus() are preferred.

Commenting Style

- **Purpose-Driven:** Comments explain the “why” of complex logic (e.g., distance calculations) rather than self-explanatory code.
- **Standard Tags:** // is used for short comments and /* ... */ for longer explanations.

Indentation and Code Structure

- **Consistency:** Standard 2-space indentation is applied across all React Native and Firebase files.
- **Brace Discipline:** All control structures (if, for, try-catch) must use explicit opening and closing braces.

File and Folder Naming

- **Identical Naming:** Files are given meaningful names based on their purpose, e.g., ServiceRequestService.js.
- **Layered Structure:** Projects are organized into logical directories: /components, /screens, /services, and /utils.

Separation of Concerns

- **UI Layer:** UI layers handle presentation, while service layers manage Firebase data operations.
- **Service Layer:** Service modules handle Firebase operations and application logic.

Scoping and Accessibility

- **Limited Scope:** Methods and variables are kept private or local unless external access is explicitly required for system functionality.

Cohesion and Coupling

- **High Cohesion:** Related functionality is grouped within the same module.
- **Low Coupling:** Dependencies between unrelated modules are minimized to improve maintainability.

DRY Principle (Don’t Repeat Yourself)

- **Code Reuse:** Repeated functionality is placed in reusable components, services, or utility functions whenever possible.

Single Responsibility Principle (SRP)

- **Focused Modules:** Each component, function, or service should have a clear and focused responsibility.

Extensibility and Maintainability

- **Open/Closed Design:** The application structure allows new services and features to be added with minimal changes to existing functionality.

Exception Handling and Logging

- **Firebase:** Firebase, authentication, and location-related operations use appropriate error handling.
- **Try-Catch:** try-catch blocks are used for asynchronous operations where errors may occur.
- **Errors:** Meaningful error messages and logs are used to simplify debugging.

Appendix C: Application Prototype

Prototype Overview

The application prototype provides a visual representation of the QuickFix system's interface and functionality. It covers the following core modules:

- Customer Module
- Service Provider Module
- Admin Module
- Registration and Login
- Provider Search and Service Selection
- Service Requests and Checkout
- Real-Time Location Tracking
- Service Management
- Profile Management
- Notifications and Feedback

Tool Used for Prototype

- **Frontend:** React Native for cross-platform mobile application development.
- **Authentication:** Firebase Authentication for secure user authentication.
- **Database:** Cloud Firestore for storing application data.
- **Storage:** Firebase Storage for storing profile images.
- **Maps:** OpenStreetMap for location-based services and map display.
- **Notifications:** Firebase Cloud Messaging (FCM) for push notifications.
- **Version Control:** GitHub for source code and repository management.

Screenshots of Key Interfaces

The key interfaces of the QuickFix application are already presented in **[3.5]**; therefore, they are not repeated in this appendix.

Navigation Flow

Here's the typical flow of the application.

- **Login/Register:** → Role Identification → Respective Dashboard → Access Features → Logout
- **Customer Path:**
  Dashboard → Select Category → Find Provider → Select Services → Checkout → Submit Request → Track Request → Feedback
- **Provider Path:**
  Dashboard → Set Availability → Manage Services → Receive Request → Accept/Reject Request → Complete Service
- **Admin Path:**
  Dashboard →Pending Applications→ Providers List → Customers List → Customer Feedbacks

Responsive Design

The application interface is designed to support different mobile screen sizes using responsive React Native layouts.